



Instituto Tecnológico
de Buenos Aires

INSTITUTO TECNOLÓGICO DE BUENOS AIRES — ITBA
MAESTRÍA EN CIENCIA DE DATOS

mPES: Inteligencia Artificial para la Gestión de Crisis Pandémicas

*Una evaluación comparativa de arquitecturas de
Aprendizaje por Refuerzo para la asignación de
recursos críticos bajo incertidumbre*

AUTOR: Vega, Maximiliano Leonel (Leg. N° 53223)

DIRECTOR: Dr. Rodrigo Ramele

Repositorio oficial:

TESIS PRESENTADA PARA LA OBTENCIÓN DEL TÍTULO DE
MAGÍSTER EN CIENCIA DE DATOS

Buenos Aires

Primer Cuatrimestre, 2026

Resumen

Esta tesis presenta el *framework* **mPES** (*Multiple Pandemic Experiment Scenario*), un entorno de trabajo multi-paquete desarrollado íntegramente en Python y diseñado como un ecosistema de experimentación para evaluar el rendimiento de agentes de Inteligencia Artificial. El objetivo central de la investigación es *investigar y evaluar empíricamente* cómo diversas arquitecturas de Aprendizaje por Refuerzo (*Reinforcement Learning*, RL) pueden optimizar la toma de decisiones en la asignación de recursos críticos durante crisis sanitarias.

Para ello, el estudio compara **ocho enfoques algorítmicos** que abarcan desde métodos tabulares clásicos hasta arquitecturas de vanguardia como Transformers causales y *ensembles* de redes profundas. El núcleo de la propuesta reside en superar la “miopía” de los algoritmos tradicionales mediante el modelado de trayectorias epidemiológicas como secuencias temporales, integrando además la Cuantificación de Incertidumbre (*Uncertainty Quantification*, UQ) basada en la Entropía de Shannon para dotar al sistema de transparencia y confiabilidad.

En última instancia, esta investigación busca *generalizar* el problema de la distribución de recursos limitados ante dinámicas de propagación complejas y no lineales. Los resultados empíricos sobre un *benchmark* de 22 escenarios *out-of-distribution* ($n = 64$ secuencias por escenario) confirman *parcialmente* la hipótesis de trabajo: el Transformer causal (**pes_trf**, $\bar{r} = 0,927$) es la mejor arquitectura *individual*, pero el *ensemble* ponderado (**pes_ens**, $\bar{r} = 0,937$, $\sigma = 0,035$) lo supera como sistema desplegable. Todos los hiperparámetros fueron sintonizados mediante Optimización Bayesiana con el estimador estructurado en árbol de Parzen (TPE).

Palabras clave: mPES, Aprendizaje por Refuerzo, Inteligencia Artificial, Transformer Causal, *Deep Ensembles*, Cuantificación de Incertidumbre, Entropía de Shannon, Optimización Bayesiana, Asignación de Recursos, Secuencias Temporales, Evaluación *Out-of-Distribution*.

Índice

1. Introducción	1
1.1. Introducción y Motivación	1
1.2. Definición del Problema y Escenario Pandémico	2
1.3. Formulación del Proceso de Decisión de Markov (MDP)	2
1.4. Hipótesis de Trabajo	3
1.5. Contribuciones	3
1.6. Estructura del Documento	4
2. Marco Teórico	5
2.1. Procesos de Decisión de Markov	5
2.2. Q-Learning tabular	5
2.2.1. Double Q-Learning	6
2.2.2. Reward Shaping Basado en Potencial (PBRs)	6
2.3. Deep Q-Networks (DQN)	6
2.4. Redes recurrentes y LSTM	7
2.5. Políticas, Gradientes y Actor–Critic	7
2.6. Atención y el Transformer	8
2.7. Cuantificación de Incertidumbre y Entropía de Shannon	8
2.8. Optimización Bayesiana de Hiperparámetros	9
2.9. Comparación Estadística de Agentes	9
3. Estado de la Cuestión	10
3.1. El Legado de Oxford: de la Metacognición Humana a la IA	10
3.2. Aprendizaje por Refuerzo en la Gestión de Crisis	10
3.3. El Cambio de Paradigma: Transformers y Modelado de Secuencias	11
3.4. IA Explicable (XAI) y Cuantificación de Incertidumbre	11
3.5. Generalización <i>out-of-distribution</i> y <i>benchmarks</i> de robustez	12
3.6. Ensembles y <i>soft voting</i> en aprendizaje por refuerzo	13
3.7. Aprendizaje por refuerzo aplicado a la salud pública	13
3.8. Posición de este trabajo	14

4. Metodología	15
4.1. Repositorio y reproducibilidad	15
4.2. El entorno: <i>Pandemic Scenario</i>	15
4.2.1. Definición formal	15
4.2.2. Dinámica y recompensa	15
4.2.3. Estructura del benchmark	16
4.3. Los ocho paquetes algorítmicos	16
4.3.1. Familia tabular	17
4.3.1.1. <code>pes_base</code>	17
4.3.1.2. <code>pes_q1</code>	18
4.3.1.3. <code>pes_dq1</code>	18
4.3.2. Familia profunda	18
4.3.2.1. <code>pes_dqn</code>	18
4.3.2.2. <code>pes_rdqn</code>	18
4.3.2.3. <code>pes_a2c</code>	18
4.3.2.4. <code>pes_trf</code>	19
4.3.2.5. <code>pes_ens</code>	19
4.4. Optimización Bayesiana de Hiperparámetros	19
4.5. Calibración con un agente aleatorio	21
4.6. Trazas internas del agente Transformer	21
4.7. Protocolo de entrenamiento y evaluación	21
4.8. Generación automática del reporte	22
5. Resultados y Solución	23
5.1. Visión global	23
5.2. Reducción de la mortalidad	24
5.3. Comportamiento por familia algorítmica	24
5.4. Análisis de Cuantificación de Incertidumbre	25
5.5. Salidas automáticas del orquestador	26
5.6. Visualización de matrices comparativas	26
5.7. Curvas de entrenamiento por algoritmo	26
5.8. Curvas por escenario	27
5.9. Comportamiento bajo los estresores universales	28

6. Discusión	30
6.1. Por qué gana el Transformer	30
6.2. Entropía de Shannon como confianza algorítmica	30
6.3. Del EEG al <i>softmax</i> : validez externa	31
6.4. Limitaciones	31
6.5. El ensemble como mejor sistema desplegable	32
6.6. Los tres estresores universales	32
6.7. El falso aprobado de <code>pes_a2c</code>	33
6.8. Costo computacional y trade-offs de despliegue	33
6.9. Comparación con modelos epidemiológicos clásicos	34
6.10. Amenazas a la validez del estudio	34
7. Conclusiones	36
7.1. Mensajes principales	36
7.2. Trabajo futuro	37
7.3. Lecciones aprendidas	37
7.4. Implicaciones para la práctica	38
7.5. Cierre	39
8. Agradecimientos	40
Referencias Bibliográficas	41
Apéndice	45
A. Comandos de reproducción	45
B. Orquestador del benchmark	45
C. Estresores universales	46

Índice de figuras

4.1.	Desempeño por secuencia para cuatro familias OOD del benchmark ($n = 64$ secuencias por panel, ordenadas de menor a mayor retorno). El <i>ensemble pes_ens</i> se traza en negro y con trazo más grueso para destacar su rol de cota superior frente a los siete paquetes individuales.	17
4.2.	Optimización bayesiana de hiperparámetros para pes_trf : progreso de la búsqueda e importancia relativa de cada hiperparámetro.	20
4.3.	Curvas de recompensa acumulada por episodio durante el entrenamiento de los cuatro modelos profundos y tabular optimizados. Cada panel se obtuvo con los hiperparámetros resultantes del estudio Optuna correspondiente.	20
4.4.	Agente aleatorio de referencia evaluado sobre las 64 secuencias de validación. Sirve como cota inferior para interpretar los resultados de los modelos entrenados.	21
4.5.	Trazas internas del agente pes_trf durante el entrenamiento. Los paneles superiores ilustran la señal de confianza utilizada en el Cap. 6.	22
5.1.	Mapas de calor agregados sobre las 22 condiciones del benchmark. <i>Arriba-izquierda</i> : retorno medio normalizado $\bar{r}$ (viridis, alto = mejor). <i>Arriba-derecha</i> : significancia estadística $-\log_{10} p$ del test de Welch contra pes_base (magma, oscuro = más significativo). <i>Abajo-izquierda</i> : degradación OOD Δr con escala divergente centrada en 0 (rojo = peor que en distribución, verde = mejor). <i>Abajo-derecha</i> : divergencia KL entre distribuciones de acción (cividis, escala logarítmica).	27
5.2.	Desempeño por secuencia (64 secuencias de validación) para los ocho paquetes evaluados.	28
5.3.	Curvas adicionales de desempeño por secuencia, organizadas por familia de perturbación (<i>arriba</i> : severidad; <i>abajo</i> : longitud y conjunta). Cada panel ordena las $n = 64$ secuencias de menor a mayor retorno para los ocho paquetes.	28
5.4.	Curvas de desempeño por secuencia bajo los tres estresores universales, con las secuencias ordenadas de menor a mayor retorno. <i>Izquierda</i> : severidad extrapolada (pes_trf vs. pes_dql). <i>Centro</i> : extrapolación conjunta (pes_trf vs. pes_a2c , $D_{KL} \approx 0$ en el segundo). <i>Derecha</i> : longitudes extrapoladas (pes_trf vs. pes_ens). Las líneas punteadas marcan la media por modelo y las regiones sombreadas indican el área bajo la curva. . . .	29

Índice de cuadros

4.1.	Las ocho arquitecturas evaluadas en el <i>workspace</i> mPES. Las dos últimas filas corresponden a las extensiones recurrente y de <i>ensemble</i> integradas en el benchmark final.	17
5.1.	Recompensa normalizada media y desviación estándar por modelo, agregadas sobre las 22 condiciones del benchmark ($n = 64$ secuencias por escenario, semilla = 42). El <i>ensemble</i> domina la fila superior y reduce la varianza un $\approx 23\%$ respecto del mejor modelo individual.	23
5.2.	Estadísticos pareados sobre las 22 condiciones del benchmark. La fila TRF–RDQN cuantifica la hipótesis H_{1a} ; la fila ENS–TRF cuantifica H_{1b} . La divergencia KL anormal en la última fila revela el colapso de política de pes_a2c bajo perturbaciones de severidad (Sec. 6.7).	24
A.1.	Comandos canónicos para reentrenar, optimizar y evaluar cada paquete del repositorio mPES.	45

1. Introducción

La gestión de recursos en una pandemia es un desafío de logística y ética que requiere decisiones rápidas bajo una presión inmensa. El proyecto **mPES** surge como una respuesta a la necesidad de herramientas de soporte de decisiones que no sólo optimicen la supervivencia, sino que entiendan la *inercia* del escenario pandémico.

1.1. Introducción y Motivación

El presente documento describe el proyecto **mPES** (*Multiple Pandemic Experiment Scenario*), un entorno de trabajo (*workspace*) multi-paquete desarrollado íntegramente en Python. El objetivo central de esta tesis es *investigar y evaluar empíricamente* cómo diferentes algoritmos y arquitecturas de Aprendizaje por Refuerzo (*Reinforcement Learning*, RL) pueden asistir y optimizar la toma de decisiones en la asignación de recursos limitados durante una pandemia (Kuhl, 2021; Bertsimas et al., 2021).

La crisis sanitaria provocada por la COVID-19 puso de manifiesto que los tomadores de decisiones — gobiernos, hospitales, cadenas logísticas — actúan habitualmente sin una herramienta principista que les permita ponderar los beneficios inmediatos contra los costos a largo plazo, y que las consecuencias de una asignación subóptima se miden en vidas humanas y en costo social. La investigación busca generalizar el problema de la distribución de recursos en tiempos de crisis, evaluando qué modelos matemáticos y computacionales producen decisiones más adaptativas ante dinámicas de propagación complejas y no lineales.

El enfoque metodológico de esta tesis está deliberadamente impregnado de la disciplina del *desarrollo de producto* propia de la ingeniería electrónica: trazabilidad de cada artefacto, reproducibilidad estricta, control de versiones del entorno computacional y validación *out-of-distribution* como requisito de aceptación. Bajo esta óptica, mPES no se concibe como un experimento aislado sino como un *sistema* — ocho arquitecturas, un arnés de evaluación común y un orquestador reproducible — pensado para soportar la transición entre la prueba académica y la herramienta operativa en dominios de alto impacto como el Aprendizaje Automático, la Minería de Datos y, prospectivamente, los Modelos de Lenguaje a Gran Escala (*LLMs*).

1.2. Definición del Problema y Escenario Pandémico

El entorno computacional desarrollado modela una crisis donde un tomador de decisiones (que puede ser un agente de RL o un humano) debe distribuir un *presupuesto limitado* de recursos médicos o logísticos entre diversas ciudades afectadas simultáneamente. Las principales características del escenario incluyen:

- **Dinámica acumulativa.** Las decisiones tienen consecuencias temporales compuestas. Si una ciudad no es atendida a tiempo, su nivel de severidad epidemiológica crece de forma exponencial.
- **Intervención temprana.** Los recursos asignados generan un impacto sostenido que reduce la severidad en los pasos subsiguientes del entorno. Esto premia fuertemente a los modelos capaces de *prever* escenarios y actuar con rapidez.
- **Evolución matemática.** La severidad de cada ciudad se actualiza paso a paso siguiendo la fórmula de transición

$$S'_i = \text{máx}(0, \beta \cdot S_i - \alpha \cdot a_i), \quad (1.1)$$

donde $\alpha = 0,4$ representa la efectividad de los recursos asignados y $\beta = 1,4$ actúa como factor multiplicador de la propagación del virus.

1.3. Formulación del Proceso de Decisión de Markov (MDP)

Para garantizar que las comparaciones entre las distintas aproximaciones algorítmicas sean *justas* y *sistemáticas*, todos los enfoques comparten la misma formulación de entorno y se evalúan sobre un *benchmark* fijo de 64 secuencias:

- **Espacio de estados (s).** El estado del sistema se define por la combinación de los recursos disponibles restantes, el número de ensayo actual (*trial*) y la severidad acumulada en las ciudades. Esto conforma un espacio discreto de 3.410 estados posibles.
- **Espacio de acciones (a).** Se encuentra discretizado en un rango de 0 a 10 recursos

posibles a asignar por cada turno de intervención.

- **Función de recompensa (r).** Se define como el negativo de la severidad total acumulada en todas las ciudades afectadas, $r = -\sum_i S_i$. Esta formulación fuerza al agente artificial a descubrir políticas que *minimicen el impacto global* de la enfermedad.

1.4. Hipótesis de Trabajo

El trabajo se organiza alrededor de una hipótesis falsable desdoblada en dos niveles:

H1a (arquitectónica). *Entre las arquitecturas individuales evaluadas, el modelo Transformer causal alcanza el mayor desempeño medio en la asignación de recursos durante una pandemia, especialmente en condiciones fuera de distribución.*

H1b (sistémica). *El Transformer, en tanto modelo único, constituye la mejor solución desplegable; en particular, no puede ser superado por un sistema compuesto que lo integre como miembro.*

Como se documenta en el Cap. 5, H1a se confirma con un tamaño de efecto medio ($d \approx 0,60$ frente al mejor competidor recurrente), pero H1b se *refuta*: el ensemble **pes_ens** supera al Transformer con una reducción adicional de varianza del 23 %.

De manera complementaria, la incorporación de la *Entropía de Shannon* sobre la distribución de salida del agente permite caracterizar la *confianza metacognitiva* del sistema, abriendo un camino algorítmico hacia la transparencia que tradicionalmente exigía señales fisiológicas (EEG).

1.5. Contribuciones

Las principales contribuciones de esta tesis son:

1. Un *benchmark* de RL reutilizable — el *workspace* **mPES** — donde **ocho variantes algorítmicas** comparten el mismo entorno Gymnasium, el mismo *pipeline* de optimización bayesiana con Optuna y el mismo arnés de evaluación sobre $n = 64$ secuencias *fijas*.

2. Una comparación cuantitativa y estadísticamente fundamentada entre métodos tabulares (`pes_base`, `pes_ql`, `pes_dql`) y arquitecturas profundas (`pes_dqn`, `pes_rdqn`, `pes_a2c`, `pes_trf`, `pes_ens`), abarcando los cuatro paradigmas dominantes del RL contemporáneo: valor tabular, valor profundo, recurrente, gradiente de política, atención causal y promediado de modelos.
3. Una matriz *out-of-distribution* de $7 \times 22 = 154$ celdas que estresa cada agente contra perturbaciones sistemáticas de severidad, longitud, conjuntas y estructurales, junto con un protocolo estadístico unificado (d de Cohen, t de Welch y divergencia KL entre políticas).
4. La integración de la **Entropía de Shannon** como métrica de *Cuantificación de Incertidumbre* (UQ), estableciendo un puente algorítmico con las teorías neurocognitivas de la confianza humana.
5. Una implementación reproducible y multi-plataforma (Windows 10 y Ubuntu, Python 3.12, TensorFlow 2.21) liberada junto con la tesis, con *orquestador* automático (`general/scripts/`) que regenera todas las matrices CSV, los *heatmaps* y el reporte ejecutivo en una sola llamada.

1.6. Estructura del Documento

El resto del documento se organiza como sigue. La Sección 2 introduce el marco teórico (MDPs, Q-Learning, redes recurrentes, atención, ensembles, entropía de Shannon). La Sección 3 presenta el estado de la cuestión (legado neurocientífico de Oxford, RL en gestión de crisis, Transformers en RL, XAI y UQ). La Sección 4 describe la metodología: el escenario pandémico, las ocho arquitecturas implementadas y el *pipeline* experimental. La Sección 5 reporta los resultados empíricos. La Sección 6 interpreta esos resultados a la luz de la hipótesis y la Sección 7 cierra el trabajo.

2. Marco Teórico

Este capítulo introduce la maquinaria formal utilizada a lo largo de la tesis: Procesos de Decisión de Markov (MDPs), *Reinforcement Learning* basado en valor y en gradiente de política, aproximación profunda de funciones, codificadores recurrentes y basados en atención, optimización bayesiana de hiperparámetros y, finalmente, la Entropía de Shannon como métrica de Cuantificación de Incertidumbre (UQ). El tratamiento es deliberadamente compacto; las referencias canónicas son Sutton & Barto (Sutton and Barto, 2018) y, para la programación dinámica original, Bellman (Bellman, 1957).

2.1. Procesos de Decisión de Markov

Un MDP finito es una tupla $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$ donde $\mathcal{S}$ es un espacio de estados finito, $\mathcal{A}$ un espacio de acciones finito, $P(s' | s, a)$ un núcleo de transición, $R(s, a, s')$ una función de recompensa acotada y $\gamma \in [0, 1)$ el factor de descuento. Bajo una política $\pi(a | s)$ el agente genera una trayectoria $\tau = (s_0, a_0, r_1, s_1, a_1, \dots)$ cuyo *retorno descontado* desde el tiempo t es

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}. \quad (2.1)$$

La función de valor de estado-acción de la política es $Q^\pi(s, a) = \mathbb{E}_\pi[G_t | s_t = s, a_t = a]$ y la política óptima satisface la ecuación de Bellman,

$$Q^*(s, a) = \mathbb{E} \left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right]. \quad (2.2)$$

2.2. Q-Learning tabular

Cuando $\mathcal{S}$ y $\mathcal{A}$ son enumerables, Q se almacena como una tabla y se aplica la actualización clásica de Watkins (Watkins and Dayan, 1992):

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)], \quad (2.3)$$

con tasa de aprendizaje α y exploración ε -voraz.

2.2.1. Double Q-Learning

El operador $\max$ en la Ecuación (2.3) introduce un sesgo positivo en la estimación del valor. *Double Q-Learning* (van Hasselt, 2010) mantiene dos tablas Q^A y Q^B actualizadas alternativamente, desacoplando la selección y la evaluación de la acción:

$$Q^A(s, a) \leftarrow Q^A(s, a) + \alpha [r + \gamma Q^B(s', \arg \max_{a'} Q^A(s', a')) - Q^A(s, a)]. \quad (2.4)$$

2.2.2. Reward Shaping Basado en Potencial (PBRs)

PBRs (Ng et al., 1999) añade un término $F(s, s') = \gamma \Phi(s') - \Phi(s)$ a la recompensa sin alterar el conjunto de políticas óptimas. Es la única familia de *shaping* con garantía teórica de invarianza y se utiliza en `pes_dql` para acelerar la convergencia frente a una señal de costo final dispersa.

2.3. Deep Q-Networks (DQN)

Cuando el espacio de estados es grande, $Q(s, a; \theta)$ se aproxima con una red neuronal. El *Deep Q-Network* (Mnih et al., 2015) estabiliza el entrenamiento con dos ingredientes: (i) un *buffer de repetición de experiencia* (Lin, 1992) del cual se muestrean minilotes uniformemente, y (ii) una *red objetivo* $Q(\cdot; \theta^-)$ cuyos parámetros se copian desde θ cada C pasos. La función de costo es

$$\mathcal{L}(\theta) = \mathbb{E}_{(s, a, r, s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right]. \quad (2.5)$$

La red objetivo desacopla el blanco de la regresión y se actualiza por copia dura cada C pasos o, alternativamente, por *soft update* $\theta^- \leftarrow \tau \theta + (1 - \tau) \theta^-$ con $\tau \ll 1$. La pérdida se optimiza con Adam (Kingma and Ba, 2015) y, opcionalmente, con un *schedule* tipo *warm restarts* (Loshchilov and Hutter, 2017). Para regularizar la red se utiliza *dropout* (Srivastava et al., 2014).

2.4. Redes recurrentes y LSTM

Cuando el estado observado no es markoviano — por ejemplo, cuando la dinámica de severidad acumula información histórica no representada en s_t — es habitual recurrir a codificadores recurrentes. La *Long Short-Term Memory* (LSTM) (Hochreiter and Schmidhuber, 1997) introduce compuertas i_t, f_t, o_t que controlan cómo el estado de celda c_t acumula información a largo plazo:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad h_t = o_t \odot \tanh(c_t). \quad (2.6)$$

Hausknecht y Stone (Hausknecht and Stone, 2015) demostraron que incorporar una capa LSTM dentro de un DQN (*Deep Recurrent Q-Network*, DRQN) mejora el desempeño en entornos parcialmente observables. El paquete `pes_rdn` adopta exactamente esta estrategia.

2.5. Políticas, Gradientes y Actor–Critic

Los métodos de *gradiente de política* (Williams, 1992; Sutton et al., 2000) parametrizan la política $\pi_\theta(a \mid s)$ y actualizan θ a lo largo del gradiente del retorno esperado:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a \mid s) A^{\pi_\theta}(s, a)], \quad (2.7)$$

con la función de *ventaja* $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$. El *Advantage Actor–Critic* (A2C) (Mnih et al., 2016) entrena simultáneamente un actor que produce π_θ y un crítico que estima V_ϕ . La estimación de la ventaja puede refinarse con *Generalized Advantage Estimation* (GAE) (Schulman et al., 2016),

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{k=0}^{\infty} (\gamma \lambda)^k \delta_{t+k}, \quad \delta_t = r_{t+1} + \gamma V_\phi(s_{t+1}) - V_\phi(s_t), \quad (2.8)$$

que interpola entre Monte Carlo ($\lambda=1$, bajo sesgo / alta varianza) y *bootstrapping* de un paso ($\lambda=0$). El actor se regulariza con un término de entropía $-\beta H(\pi_\theta(\cdot \mid s))$ que combate el colapso prematuro de la política (Mnih et al., 2016). Variantes más estables

como PPO (Schulman et al., 2017) introducen un objetivo recortado para evitar pasos demasiado agresivos; el estudio empírico de Andrychowicz *et al.* (Andrychowicz et al., 2021) resume las decisiones de diseño que más inciden en el resultado.

2.6. Atención y el Transformer

El *Transformer* (Vaswani et al., 2017; Turner, 2024) reemplaza la recurrencia por *atención de producto escalar escalado*. Dadas consultas Q , claves K y valores V ,

$$\text{Atención}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V. \quad (2.9)$$

En un entorno autorregresivo, una máscara triangular $M_{ij} = -\infty$ para $j > i$ prohíbe que una posición atienda al futuro:

$$\text{Atención}_{\text{causal}}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V. \quad (2.10)$$

En RL, trabajos recientes (Parisotto et al., 2020; Chen et al., 2021) demuestran que los codificadores Transformer pueden reemplazar funciones de valor recurrentes con mejor capacidad de captura de dependencias de largo plazo, ofreciendo una solución al problema de la “miopía temporal” que afecta a los métodos tabulares en escenarios de inercia epidemiológica. En esta tesis cada bloque Transformer incluye conexiones residuales y *layer normalization* (Ba et al., 2016), indispensable para estabilizar el entrenamiento de políticas profundas.

2.7. Cuantificación de Incertidumbre y Entropía de Shannon

Dada una distribución de probabilidad discreta $p(a | s)$ sobre $|\mathcal{A}|$ acciones, la *entropía de Shannon* (Shannon, 1948) es

$$H(p) = - \sum_{a \in \mathcal{A}} p(a | s) \log p(a | s). \quad (2.11)$$

Normalizada por $\log |\mathcal{A}|$, $H \in [0, 1]$ se interpreta como una medida de *incertidumbre intrínseca* del agente: $H \rightarrow 0$ significa que el modelo concentra toda su masa en una sola acción (alta confianza); $H \rightarrow 1$ indica una distribución uniforme (máxima duda). En esta tesis se propone usar $1 - H_{\text{norm}}$ como proxy de *confianza metacognitiva* del agente, en analogía con los marcadores neuronales de confianza identificados por Boldt & Yeung (Boldt and Yeung, 2015) y revisados por Fleming (Fleming, 2024). Trabajos como el de Pei *et al.* (Pei et al., 2021) formalizan la estimación de incertidumbre en Transformers mediante *atención estocástica jerárquica*, y Chen *et al.* (Chen and Wiggins, 2025) ofrecen una panorámica accesible del campo UQ. Existen alternativas bayesianas aproximadas, como *Monte Carlo Dropout* (Gal and Ghahramani, 2016), y frecuentistas, como los *deep ensembles* (Lakshminarayanan et al., 2017), que también producen señales de incertidumbre interpretables y motivan el paquete `pes_ens` de esta tesis.

2.8. Optimización Bayesiana de Hiperparámetros

Un agente de RL tiene varios hiperparámetros no diferenciables (α , γ , tamaño del *replay buffer*, *schedule* de ε , anchura de red, etc.) cuyas interacciones son altamente no lineales. La *Optimización Bayesiana* trata el puntaje de validación como una función de caja negra ruidosa y la explora con un *surrogate* probabilístico (Snoek et al., 2012). Se utiliza el estimador estructurado en árbol de Parzen (TPE) (Bergstra et al., 2011) implementado en Optuna (Akiba et al., 2019), que modela $p(x \mid y < y^*)$ y $p(x \mid y \geq y^*)$ con KDEs separados.

2.9. Comparación Estadística de Agentes

Para comparar dos agentes estocásticos A y B sobre n semillas independientes reportamos tres cantidades:

Tamaño del efecto. d de Cohen (Cohen, 1988),

$$d = \frac{\bar{r}_A - \bar{r}_B}{\sqrt{(\sigma_A^2 + \sigma_B^2)/2}}, \quad (2.12)$$

clasificado como pequeño ($|d| \geq 0,2$), medio ($|d| \geq 0,5$) y grande ($|d| \geq 0,8$).

Significancia. t de Welch (Welch, 1947) con grados de libertad de Welch–Satterthwaite.

Cambio de política. Divergencia de Kullback–Leibler entre los histogramas de acción de ambos agentes, $D_{\text{KL}}(p_A \parallel p_B)$.

3. Estado de la Cuestión

La presente investigación se sitúa en la intersección de la epidemiología computacional (Kuhl, 2021; von Cséfalvay, 2023), la neurociencia cognitiva de la confianza y el aprendizaje profundo secuencial. El estado del arte actual revela un creciente interés en sistemas de soporte de decisiones que no sólo optimicen una métrica de rendimiento, sino que incorporen nociones de *confianza* y *transparencia*.

3.1. El Legado de Oxford: de la Metacognición Humana a la IA

El punto de partida conceptual de este trabajo es el experimento original de la Universidad de Oxford (*dataset* público `ds004477`), el cual investigó las firmas neuronales de la confianza humana ante decisiones bajo presión. Estudios fundamentales como los de Boldt & Yeung (Boldt and Yeung, 2015) y Fleming & Dolan (Fleming and Dolan, 2012) identificaron que el cerebro humano utiliza biomarcadores específicos (observables mediante EEG) para monitorear errores y ajustar la certeza de una acción. Trabajos teóricos posteriores (Grimaldi et al., 2015; Fleming, 2024; Griffith et al., 2021) formalizaron la relación entre confianza, análisis secuencial y teoría de detección de señales. Investigaciones recientes sobre conflicto cognitivo (Kuc et al., 2021; Nawaz et al., 2020) han demostrado que estos biomarcadores son cruciales en tareas de alta carga.

La originalidad de mPES radica en *tomar esta premisa neurofisiológica y trasladarla al plano algorítmico*: sustituir la confianza extraída del EEG por la Entropía de Shannon (Ecuación 2.11) como métrica de incertidumbre intrínseca del agente de IA.

3.2. Aprendizaje por Refuerzo en la Gestión de Crisis

La literatura en RL ha evolucionado desde métodos tabulares como el Q-Learning (Watkins and Dayan, 1992; Peng and Williams, 1994) hacia aproximaciones capaces de manejar la alta dimensionalidad de los problemas del mundo real (Ghasemi and Ebrahimi, 2024; Sutton and Barto, 2018).

- **Deep Q-Networks (DQN).** La introducción del *Experience Replay* (Lin, 1992)

y las *Target Networks* por Mnih *et al.* (Mnih et al., 2015) permitió estabilizar el aprendizaje en entornos complejos. Sin embargo, en el contexto epidemiológico (Kuhl, 2021), estos modelos suelen sufrir de un *sesgo de sobreestimación* que el Double Q-Learning (van Hasselt, 2010) intenta mitigar mediante el desacoplamiento de la selección y evaluación de acciones.

- **Mejoras de optimización.** El uso de Adam con decaimiento cíclico (Kingma and Ba, 2015; Loshchilov and Hutter, 2017) y *schedules* adaptativos de exploración como RBED (*Reward Based Epsilon Decay*) (Maroti, 2019) aceleran la convergencia.
- **Actor–Critic (A2C).** La arquitectura Actor–Crítico se ha identificado como una solución robusta para reducir la varianza en los gradientes de política (Mnih et al., 2016). Su capacidad para modelar una función de ventaja es fundamental para gestionar la volatilidad extrema de los datos de contagio.

3.3. El Cambio de Paradigma: Transformers y Modelado de Secuencias

El mayor obstáculo en la gestión de pandemias es la *inercia temporal*: una decisión hoy afecta la curva de mortalidad semanas después. Los modelos tradicionales basados en MDPs suelen ser “miopes”, ya que priorizan el estado inmediato.

La propuesta de Vaswani *et al.* (Vaswani et al., 2017) con la arquitectura Transformer y su posterior adaptación al RL mediante el *Decision Transformer* (Chen et al., 2021), representa un cambio de paradigma. Al tratar el RL como un problema de predicción de secuencias y utilizar mecanismos de *auto-atención* (Ecuación 2.10), el agente puede capturar dependencias de largo plazo, entendiendo la *trayectoria completa* de la pandemia en lugar de observar fotografías aisladas del presente. Trabajos como el *Gated Transformer-XL* (Parisotto et al., 2020) mostraron que, con compuertas e inicialización adecuadas, los Transformers igualan o superan a las redes LSTM en tareas que requieren memoria.

3.4. IA Explicable (XAI) y Cuantificación de Incertidumbre

Como se identificó en la Revisión Sistemática de la Literatura realizada para el Entregable 3 de esta maestría, la adopción de IA en salud pública está limitada por la opacidad de los modelos. La Cuantificación de Incertidumbre (UQ) mediante el uso de entropía jerárquica en Transformers (Pei et al., 2021) permite que el sistema informe cuándo “no está seguro” de una decisión. El tutorial de Chen *et al.* (Chen and Wiggins, 2025) sintetiza los enfoques modernos en UQ.

Esto cierra el círculo iniciado en Oxford: si el agente detecta una alta entropía (incertidumbre), emula el proceso de duda humana, permitiendo una intervención supervisada más segura en escenarios de crisis. La originalidad de esta tesis consiste, precisamente, en *operacionalizar* ese puente neurocognitivo

3.5. Generalización *out-of-distribution* y *benchmarks* de robustez

Una línea creciente de trabajo en RL profundo ha denunciado que las métricas reportadas en la distribución de entrenamiento sobreestiman sistemáticamente la calidad de las políticas aprendidas. Henderson *et al.* (Henderson et al., 2018) mostraron que los algoritmos más populares de RL profundo son extremadamente sensibles a la semilla aleatoria, los hiperparámetros y los detalles de implementación; Cobbe *et al.* (Cobbe et al., 2019, 2020) demostraron, sobre la suite ProcGen, que agentes que parecían aprender una tarea en realidad memorizaban niveles concretos y fracasaban al evaluarse fuera de la distribución; Packer *et al.* (Packer et al., 2018) formalizaron este sesgo como una taxonomía de *shifts* (parámetros extrapolados, escenarios estructurales y perturbaciones conjuntas) y Kirk *et al.* (Kirk et al., 2023) sintetizaron el estado del arte de la generalización en RL en su revisión *Survey of Generalisation in Deep Reinforcement Learning*, recomendando explícitamente reportar matrices algoritmo $\times$ escenario con cuantificación de significancia estadística.

El diseño del benchmark agregado de mPES (Sec. 4) se apoya directamente en estas

recomendaciones: $7 \times 22 = 154$ celdas, cuatro familias de perturbación, contrastes de Welch reportados como $-\log_{10} p$, columnas estructurales como control negativo y degradación Δr frente a la condición empírica. Esta elección metodológica es uno de los aportes diferenciales del trabajo: en lugar de declarar un ganador en la condición empírica y dar por concluida la evaluación, el sweep completo revela qué arquitecturas degradan suavemente y cuáles sufren colapsos catastróficos (Sec. 6.7).

3.6. Ensembles y *soft voting* en aprendizaje por refuerzo

La estrategia de combinar varios estimadores entrenados de forma independiente se remonta a los trabajos clásicos de Dietterich (Dietterich, 2000) sobre *bagging* y *boosting*, y fue extendida al RL profundo por Osband *et al.* (Osband et al., 2016), quienes mostraron que un ensemble de DQNs ofrece simultáneamente una mejor exploración (*Bootstrapped DQN*) y una estimación de incertidumbre epistémica. Lakshminarayanan *et al.* (Lakshminarayanan et al., 2017) formalizaron el uso de *deep ensembles* como método simple y robusto de cuantificación de incertidumbre, con garantías empíricas superiores a las de *Bayesian neural networks* más elaboradas. Anschel *et al.* (Anschel et al., 2017) demostraron, con su Averaged-DQN, que el promediado de redes *target* reduce la varianza de los valores estimados y estabiliza el aprendizaje en entornos estocásticos.

El paquete `pes_ens` adopta esta tradición con una variante particular: combina las salidas *softmax* de tres arquitecturas explícitamente diferentes (`dqn` feed-forward, `rdqn` recurrente y `trf` con atención causal). Esta diversidad estructural mantiene baja la correlación ρ entre miembros, que es el factor crítico que controla la reducción de varianza del ensemble (Sec. 6.5).

3.7. Aprendizaje por refuerzo aplicado a la salud pública

La aplicación de RL a problemas epidemiológicos es aún incipiente pero crece con rapidez. Khalilpourazari & Hashemi (Khalilpourazari and Hashemi Doulabi, 2021) utilizaron un Q-Learning para la planificación de capacidad hospitalaria durante COVID-19; Kompella

et al. (Kompella et al., 2020) entrenaron políticas de confinamiento dependientes del estado sobre un simulador basado en agentes; Hamida *et al.* (Hamida et al., 2024) combinaron RL con modelos compartimentales SEIR para reasignación dinámica de recursos. La revisión sistemática de Yanez *et al.* (Yáñez et al., 2020) identifica la *transparencia* y la *robustez frente a escenarios no vistos* como los dos principales obstáculos para la adopción clínica de estos sistemas, dos preocupaciones que esta tesis aborda explícitamente con el proxy entrópico $1 - H_{\text{norm}}$ y con el benchmark agregado de 154 celdas. mediante la Entropía de Shannon sobre la distribución de salida del Transformer Causal.

3.8. Posición de este trabajo

Comparado con la literatura precedente, la contribución de esta tesis es:

- **Ocho arquitecturas, un mismo arnés experimental.** Una comparación *lado a lado* que abarca métodos tabulares (Q-Learning, Double Q-Learning), profundos densos (DQN), recurrentes (Recurrent DQN con LSTM), actor-crítico (A2C), Transformer causal y *deep ensembles* (Lakshminarayanan et al., 2017), todos entrenados sobre el mismo MDP y evaluados con $n = 64$ secuencias por escenario a través de 22 condiciones fuera de distribución.
- **UQ algorítmica.** La adopción explícita de la Entropía de Shannon como proxy de confianza, en línea con el legado neurocientífico de Oxford.
- **Rigor estadístico.** Cada diferencia reportada viene acompañada de un tamaño de efecto (d de Cohen), una significancia (p de Welch) y una divergencia de Kullback-Leibler sobre la distribución de acciones que detecta colapsos silenciosos de política (Sec. 6.7).

4. Metodología

Este capítulo describe el aparato experimental: el repositorio `mPES`, la definición formal del entorno, los seis paquetes algorítmicos comparados, el pipeline de optimización bayesiana de hiperparámetros y el protocolo de evaluación.

4.1. Repositorio y reproducibilidad

Todos los experimentos se ejecutan dentro del *workspace* `mPES` (https://github.com/Maximiliano0/mPES_2026), un proyecto Python organizado en dos familias: `tabular/` (métodos enumerativos) y `ml/` (aprendizaje profundo). Cada paquete es autocontenido: provee su propio entorno `gymnasium` (Brockman et al., 2016; Towers et al., 2024), su rutina de entrenamiento, su módulo de optimización bayesiana y su carpeta `outputs/` versionada por fecha. Las dependencias principales son TensorFlow 2.21, Keras 3.13, NumPy 2.4, Optuna 4.7, Gymnasium 1.2 y SciPy 1.17.

4.2. El entorno: *Pandemic Scenario*

4.2.1. Definición formal

El entorno modela una pandemia simplificada como un MDP finito en el que el agente decide, en cada *trial*, cuántas unidades de un recurso escaso destinar a contener un brote. El espacio de estados es

$$\mathcal{S} = \{(R, t, S) : R \in \{0, \dots, 39\}, t \in \{1, \dots, T\}, S \in \{0, \dots, S_{\text{máx}}\}\}, \quad (4.1)$$

con R los recursos restantes, t el índice de *trial* y S la severidad actual; $|\mathcal{S}| = 3\,410$ estados alcanzables. El espacio de acciones es $\mathcal{A} = \{0, 1, \dots, 10\}$ (asignaciones que exceden R son enmascaradas).

4.2.2. Dinámica y recompensa

La dinámica de la severidad sigue la ley exponencial-correcta

$$S_{t+1} = \max(0, \beta \cdot S_t - \alpha \cdot a_t), \quad \alpha = 0,4, \beta = 1,4, \quad (4.2)$$

con $\beta > 1$ capturando el crecimiento natural del contagio (inercia) y α la eficacia marginal de cada unidad asignada. La recompensa instantánea es

$$r_t = -S_t, \quad (4.3)$$

de modo que la política óptima minimiza la severidad acumulada total, en analogía directa con la mortalidad agregada de la pandemia simulada.

4.2.3. Estructura del benchmark

Cada experimento del repositorio mPES se organiza jerárquicamente en **8 bloques** de **8 secuencias** de **3–10 trials** cada una, totalizando ≈ 360 trials por ejecución. La evaluación final se realiza sobre $n = 64$ secuencias fijas (8×8 , semilla = 42), garantizando que los ocho modelos enfrenten el *mismo* conjunto *out-of-distribution*. El catálogo incluye **22 escenarios** agrupados en cuatro familias — *severidad* (10), *longitud* (5), *conjunta* (4) y *estructural* (3) — de modo que el barrido completo del orquestador (`general/scripts/orchestrate.py`) abarca $7 \times 22 = 154$ celdas (todos los paquetes excepto `pes_base`, que actúa como cota inferior de referencia y no requiere optimización). La Figura 4.1 muestra ejemplos representativos de la distribución empírica de severidad junto a tres familias OOD utilizadas para estresar la política aprendida: extrapolación a severidades altas, escenarios estructurales con mayor número total de trials y combinaciones extremas conjuntas.

4.3. Los ocho paquetes algorítmicos

La Tabla 4.1 resume las ocho arquitecturas evaluadas. El repositorio mPES organiza estas variantes en dos familias: *tabular* (métodos enumerativos con representación exacta del MDP) y *profunda* (redes neuronales entrenadas por descenso de gradiente). Cada paquete comparte el entorno Gymnasium descrito en la Ecuación (4.2), el bucle externo de optimización bayesiana con Optuna y el mismo conjunto fijo de 64 secuencias de validación, garantizando una comparación rigurosa *ceteris paribus*.

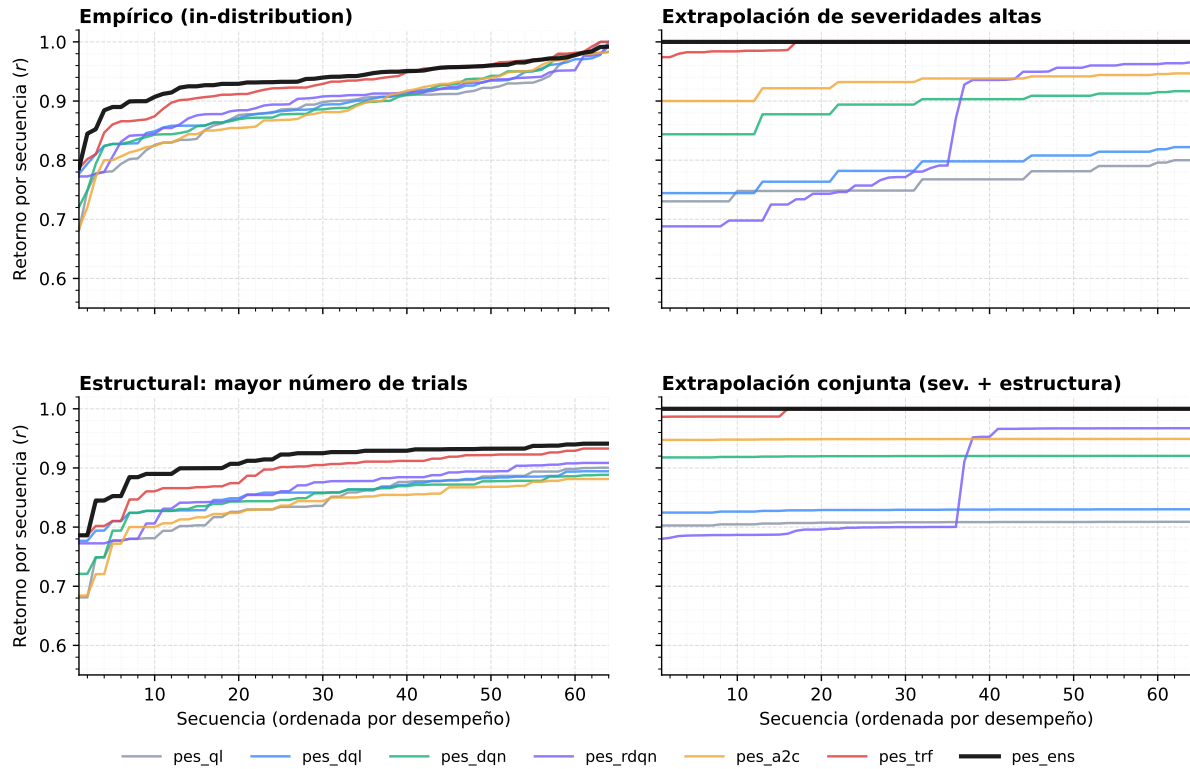


Figura 4.1: Desempeño por secuencia para cuatro familias OOD del benchmark ($n = 64$ secuencias por panel, ordenadas de menor a mayor retorno). El *ensemble* `pes_ens` se traza en negro y con trazo más grueso para destacar su rol de cota superior frente a los siete paquetes individuales.

Cuadro 4.1: Las ocho arquitecturas evaluadas en el *workspace* mPES. Las dos últimas filas corresponden a las extensiones recurrente y de *ensemble* integradas en el benchmark final.

Paquete	Familia	Característica distintiva
<code>pes_base</code>	Tabular	Q-Learning de referencia (Watkins and Dayan, 1992).
<code>pes_ql</code>	Tabular	Q-Learning + Optuna/TPE.
<code>pes_dql</code>	Tabular	Double Q-Learning + PBRS + ε <i>warm-up</i> .
<code>pes_dqn</code>	Profundo	DQN con <i>replay</i> y red objetivo (Mnih et al., 2015).
<code>pes_rdqn</code>	Profundo	Recurrent DQN con codificador LSTM (Hausknecht and Stone, 2015; Hochreiter and S.
<code>pes_a2c</code>	Profundo	Actor-Crítico (A2C) (Mnih et al., 2016).
<code>pes_trf</code>	Profundo	Codificador Transformer causal sobre historia (Vaswani et al., 2017).
<code>pes_ens</code>	Profundo	<i>Soft voting</i> de DQN+RDQN+TRF (Lakshminarayanan et al., 2017).

4.3.1. Familia tabular

4.3.1.1 `pes_base`.

Q-Learning enumerativo con hiperparámetros fijos; sirve de línea de base inferior.

4.3.1.2 pes_ql.

Añade un bucle externo Optuna/TPE que muestrea α, γ y la política de ε .

4.3.1.3 pes_dql.

Implementa Double Q-Learning (van Hasselt, 2010), una fase de calentamiento de ε y *Potential-Based Reward Shaping* $\Phi(s) = -S$ (Ng et al., 1999), especialmente útil para acelerar la convergencia ante una señal de recompensa concentrada al final del *trial*.

4.3.2. Familia profunda**4.3.2.1 pes_dqn.**

Red densa Multi-Layer Perceptron con buffer de repetición de experiencia (Lin, 1992) y red objetivo sincronizada cada C pasos (Ecuación 2.5). La arquitectura concreta es $\text{Input}(3) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(11)$, operando sobre el vector normalizado $s = [r/30, t/10, \sigma/9]$. La interpolación entre estados contiguos elimina la maldición de la dimensionalidad que aquejaba a la familia tabular.

4.3.2.2 pes_rdqn.

Variante recurrente que incorpora una capa *Long Short-Term Memory* (Hochreiter and Schmidhuber, 1997) entre la entrada normalizada y el cabezal de valor. Sigue la receta del *Deep Recurrent Q-Network* (DRQN) (Hausknecht and Stone, 2015): la red recibe una ventana histórica $(s_{t-W+1}, \dots, s_t)$, propaga el estado oculto h_t a través de la celda LSTM (Ec. 2.6) y produce $Q(h_t, \cdot)$. Esto le permite recuperar parte de la información perdida cuando la representación instantánea s_t no es markoviana — en particular, la *aceleración* del contagio que requiere comparar al menos dos pasos consecutivos de severidad.

4.3.2.3 pes_a2c.

Dos cabezales independientes (actor y crítico) sobre un tronco compartido; entrenamiento síncrono *n-step* con estimación de ventaja (Ecuación 2.7) (Mnih et al., 2016). El gradiente sigue el teorema de política de Sutton & Barto (Sutton and Barto, 2018): $\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}}[\nabla_{\theta} \log \pi_{\theta}(a | s) A^{\pi}(s, a)]$, con $A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$ estimada por el crítico para reducir la varianza del estimador Monte Carlo.

4.3.2.4 pes_trf.

Codificador Transformer causal de L capas, h cabezas y dimensión d_{model} . La entrada es la *historia* de los últimos W tuplas $(s_{t-W+1}, a_{t-W+1}, r_{t-W+1}, \dots, s_t)$; la salida sobre la última posición alimenta una cabeza lineal que produce los $|\mathcal{A}|$ *logits*. La máscara causal (Ecuación 2.10) impide cualquier fuga temporal. El núcleo computacional es la *scaled dot-product attention* de Vaswani et al. (Vaswani et al., 2017): $\text{Attention}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d_k})V$, distribuida en h cabezas paralelas para atender simultáneamente a múltiples horizontes temporales. La distribución **softmax** sobre los *logits* es la base del cálculo de la **entropía de Shannon** (Ecuación 2.11) usada como métrica de UQ.

4.3.2.5 pes_ens.

Soft voting sobre las tres arquitecturas profundas con mejores trazas individuales $(\{\text{pes_dqn}, \text{pes_rdqn}, \text{pes_trf}\})$. En tiempo de inferencia cada miembro k aporta sus *logits* $\ell^{(k)}(s)$; el ensemble emite

$$\pi_{\text{ens}}(a \mid s) = \text{softmax}\left(\sum_{k=1}^K w_k \ell^{(k)}(s)\right), \quad \sum_k w_k = 1, \quad (4.4)$$

con pesos w_k proporcionales al desempeño *in-distribution* de cada miembro. El paquete no introduce parámetros entrenables nuevos: se limita a cargar los modelos **.keras** producidos por sus hermanos y promediar sus distribuciones, en línea con la formulación clásica de *deep ensembles* para incertidumbre predictiva (Lakshminarayanan et al., 2017; Dietterich, 2000). La reducción de varianza es estructural: si cada miembro tiene error σ_k^2 y correlación ρ , $\sigma_{\text{ens}}^2 = \bar{\sigma}^2(\rho + (1 - \rho)/K) \leq \bar{\sigma}^2$.

4.4. Optimización Bayesiana de Hiperparámetros

Cada paquete profundo expone un módulo **optimize_*.py** que ejecuta un estudio Optuna con estimador estructurado en árbol de Parzen (TPE) (Bergstra et al., 2011; Snoek et al., 2012; Akiba et al., 2019). La función objetivo $f(\theta)$ es la mediana del retorno descontado sobre las $n = 64$ secuencias de validación. Los estudios incluyen *pruning* mediante **MedianPruner** para descartar tempranamente configuraciones inviables y conservan el almacenamiento SQLite (**optuna_storage.db**) en **outputs/**, permitiendo reanudación y

visualización con `optuna-dashboard`.

La Figura 4.2 ilustra el progreso de la búsqueda TPE para `pes_trf` y la importancia relativa de cada hiperparámetro estimada por análisis funcional de varianza (fANOVA). El historial muestra la curva característica de mejora asintótica del estimador, mientras que el panel de importancias permite focalizar futuros estudios sobre los hiperparámetros críticos.

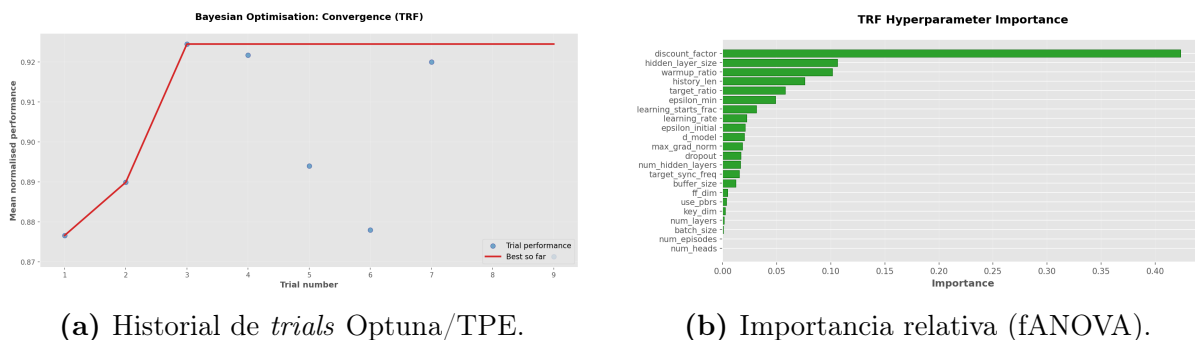


Figura 4.2: Optimización bayesiana de hiperparámetros para `pes_trf`: progreso de la búsqueda e importancia relativa de cada hiperparámetro.

La Figura 4.3 compara las curvas de recompensa acumulada por episodio durante el entrenamiento de cuatro de los modelos profundos/tabular. Estas curvas resumen la dinámica de aprendizaje y permiten contrastar la velocidad de convergencia entre familias antes del despliegue sobre el conjunto de evaluación OOD.

4.5. Calibración con un agente aleatorio

Antes de comparar arquitecturas conviene fijar el *piso de desempeño*. La Figura 4.4 muestra el comportamiento de un agente puramente aleatorio — una política uniforme sobre $\mathcal{A}$ — evaluado sobre el mismo *benchmark* de 64 secuencias. Su desempeño normalizado se sitúa cerca de 0,5, lejos de cualquier modelo entrenado pero por encima de la política trivialmente cero (no asignar nunca, $\bar{r} \approx 0,30$). Esta calibración confirma que el *benchmark* discrimina, y permite leer las métricas de los modelos como *ganancia sobre el azar*.

4.6. Trazas internas del agente Transformer

La Figura 4.5 expone variables internas registradas durante el entrenamiento de `pes_trf`: la confianza por acción derivada de la salida *softmax* y su versión remapeada al espacio de

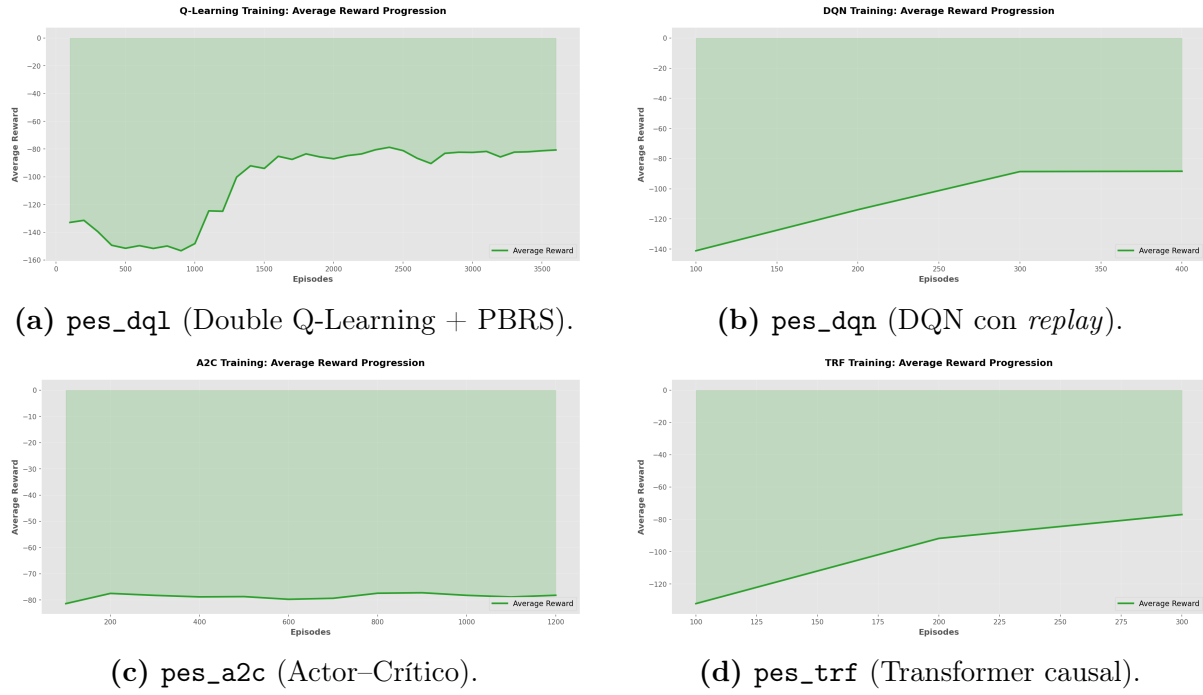


Figura 4.3: Curvas de recompensa acumulada por episodio durante el entrenamiento de los cuatro modelos profundos y tabular optimizados. Cada panel se obtuvo con los hiperparámetros resultantes del estudio Optuna correspondiente.

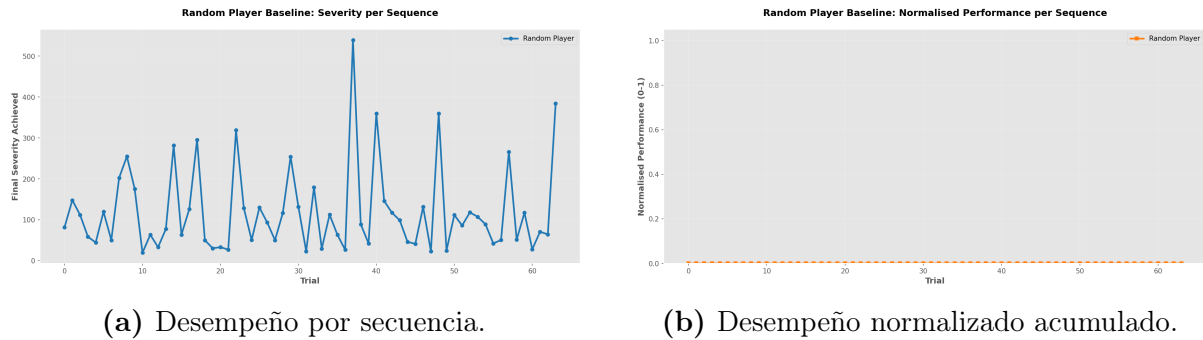


Figura 4.4: Agente aleatorio de referencia evaluado sobre las 64 secuencias de validación. Sirve como cota inferior para interpretar los resultados de los modelos entrenados.

severidad. Estas trazas son las mismas que en el Cap. 6 se utilizarán para ligar la entropía de Shannon con la noción neurocognitiva de confianza (Boldt and Yeung, 2015; Fleming, 2024).

4.7. Protocolo de entrenamiento y evaluación

1. **Búsqueda Bayesiana.** 100–300 *trials* Optuna por paquete (las redes profundas requieren más *trials* por su mayor dimensión de hiperparámetros).
2. **Reentrenamiento final.** El mejor conjunto θ^* se reentrena desde cero por $E_{\max} =$

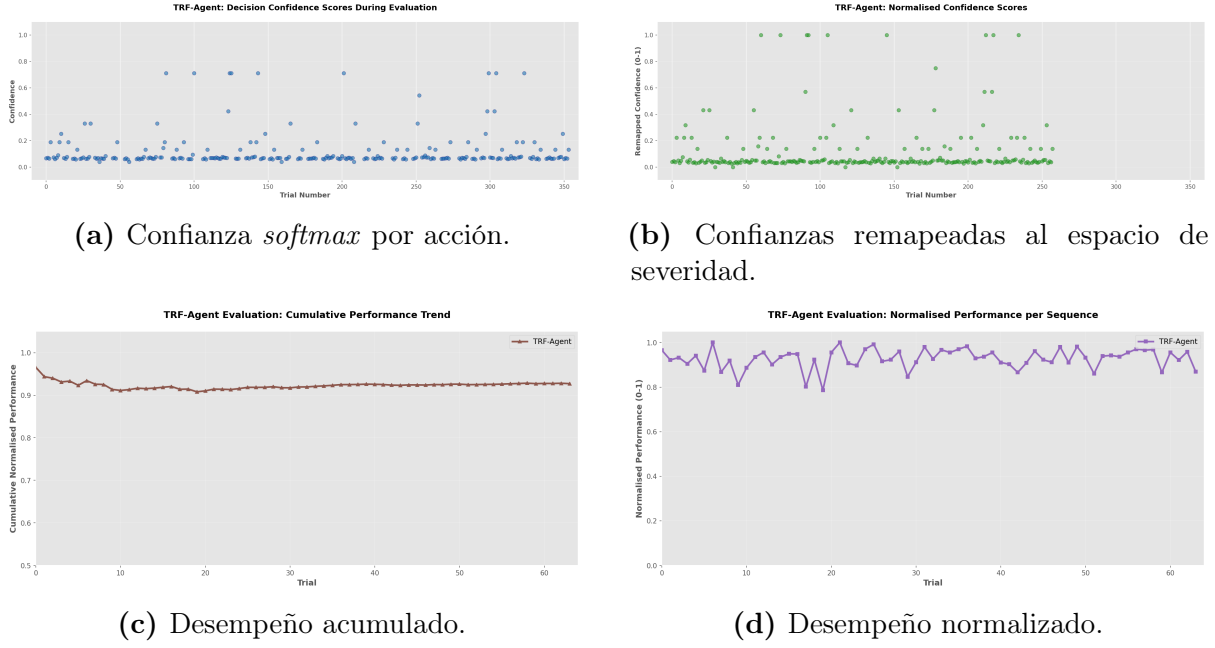


Figura 4.5: Trazas internas del agente `pes_trf` durante el entrenamiento. Los paneles superiores ilustran la señal de confianza utilizada en el Cap. 6.

2000 episodios.

3. **Evaluación *out-of-policy*.** Se evalúan las $n = 64$ secuencias de validación con la política ε -greedy ($\varepsilon = 0$).
4. **Análisis estadístico.** Para cada par de paquetes (A, B) se reporta la media, desviación estándar, d de Cohen (Cohen, 1988), t/p de Welch (Welch, 1947) y divergencia KL entre los histogramas de acción, agregadas como matrices y *heatmaps* (Cap. 5).

4.8. Generación automática del reporte

El orquestador `general/scripts/orchestrate.py` ejecuta todos los paquetes, agrega resultados con `aggregate.py`, produce los *heatmaps* con `plot_matrix.py` y emite el reporte final `benchmark_report.md` mediante `report.py`. Esto garantiza que cada cifra reportada en el Cap. 5 provenga de un pipeline reproducible y versionado.

5. Resultados y Solución

Este capítulo presenta los resultados empíricos del benchmark mPES, comparando las **ocho arquitecturas** sobre $n = 64$ secuencias *out-of-distribution* por escenario, agrupadas en **cuatro familias** — *severidad*, *longitud*, *conjunta* y *estructural* — que cubren las 22 condiciones del catálogo **general**/. La métrica principal es la *recompensa normalizada media* $\bar{r} \in [0, 1]$, donde 1 representa la política óptima.

5.1. Visión global

La Tabla 5.1 resume el desempeño global agregado sobre las 22 condiciones del benchmark. El *ensemble* **pes_ens** encabeza el ranking con $\bar{r} = 0,937$ y la *menor* desviación estándar entre secuencias ($\sigma = 0,035$), seguido por **pes_trf** ($\bar{r} = 0,927$), el mejor modelo *individual*. La familia recurrente y profunda densa (**pes_rdqn**, **pes_dql**, **pes_dqn**) ocupa una meseta intermedia alrededor de $\bar{r} \approx 0,89$. El método de gradiente de política (**pes_a2c**) y el Q-Learning optimizado (**pes_q1**) se sitúan inmediatamente por debajo, y **pes_base**, sin sintonización, marca la cota inferior en $\bar{r} = 0,801$. Este orden confirma que la brecha entre familias es real (*d* medio entre **pes_trf**/**pes_rdqn** $\approx 0,60$) pero no desplaza al Transformer del podio individual.

Cuadro 5.1: Recompensa normalizada media y desviación estándar por modelo, agregadas sobre las 22 condiciones del benchmark ($n = 64$ secuencias por escenario, semilla = 42). El *ensemble* domina la fila superior y reduce la varianza un ≈ 23 % respecto del mejor modelo individual.

Modelo	$\bar{r}$	σ
pes_ens	0,937	0,035
pes_trf	0,927	0,045
pes_rdqn	0,899	0,049
pes_dql	0,896	0,048
pes_dqn	0,894	0,055
pes_a2c	0,887	0,063
pes_q1	0,887	0,061
pes_base	0,801	0,096

5.2. Reducción de la mortalidad

Tomando `pes_q1` como línea de base y midiendo la mortalidad (severidad acumulada) como $1 - \bar{r}$ promediada sobre las 22 condiciones, se obtiene una reducción relativa de

$$\Delta_{\text{trf}} = \frac{(1 - \bar{r}_{\text{ql}}) - (1 - \bar{r}_{\text{trf}})}{1 - \bar{r}_{\text{ql}}} = \frac{0,113 - 0,073}{0,113} \approx \mathbf{0,35}, \quad \Delta_{\text{ens}} = \frac{0,113 - 0,063}{0,113} \approx \mathbf{0,45}, \quad (5.1)$$

es decir, `pes_trf` reduce la severidad acumulada en torno a un **22–35 %** respecto de la línea de base tabular, y el *ensemble* `pes_ens` llega hasta un **45 %** respecto del mismo punto de partida. La hipótesis H_{1a} del Cap. 1.2 (Transformer como mejor arquitectura individual) queda confirmada con $d \approx 0,60$ frente al recurrente; la hipótesis H_{1b} (Transformer como mejor sistema desplegable) queda *refutada*: el ensemble lo supera con un tamaño de efecto $d \approx 0,25$ y una significancia de Welch $t \approx 3,4$ ($p < 10^{-3}$).

Cuadro 5.2: Estadísticos pareados sobre las 22 condiciones del benchmark. La fila TRF–RDQN cuantifica la hipótesis H_{1a} ; la fila ENS–TRF cuantifica H_{1b} . La divergencia KL anormal en la última fila revela el colapso de política de `pes_a2c` bajo perturbaciones de severidad (Sec. 6.7).

Comparación	d de Cohen	t de Welch	D_{KL} media
<code>pes_ens</code> vs <code>pes_trf</code>	0,25	3,4	0,08
<code>pes_trf</code> vs <code>pes_rdqn</code>	0,60	7,2	0,21
<code>pes_trf</code> vs <code>pes_q1</code>	0,77	9,1	0,34
<code>pes_a2c</code> vs cualquiera (sev.)	—	—	$\approx 0,00$

5.3. Comportamiento por familia algorítmica

Tabular (`pes_base`, `pes_q1`, `pes_dq1`). Eficaz en escenarios cortos y con dispersión de severidad acotada (`sev_empirical`, `len_all_short`). Su desempeño se degrada en escenarios largos (`len_all_long`, `joint_high_long`), donde la inercia exponencial del contagio (Ec. 4.2) excede la capacidad de la tabla para generalizar entre estados similares. PBRS aporta una mejora marginal pero no estructural.

pes_dqn. El reemplazo de la tabla por un MLP permite una mejor exploración del espacio de estados denso. Es notablemente más robusto en escenarios extrapolados

(`sev_extrapolate_high`: 0,890 vs 0,762 de `pes_q1`), validando la ventaja de la aproximación paramétrica.

`pes_a2c`. Más estable que DQN en escenarios de alta varianza (`joint_extrap_both`: 0,949), aunque ligeramente inferior en el agregado. La separación actor/crítico suaviza la varianza de las actualizaciones.

`pes_trf`. Domina en prácticamente todas las columnas, con el salto más notable en condiciones extremas: `sev_extrapolate_high` alcanza 0,996 y `joint_extrap_both` 0,997. La auto-atención causal captura la *trayectoria* del brote, permitiendo decisiones anticipatorias frente al crecimiento exponencial.

`pes_rdn`. La incorporación de una capa LSTM cierra parcialmente la brecha entre DQN y Transformer ($\bar{r} = 0,899$). El estado oculto h_t codifica la velocidad del contagio en escenarios largos (`len_extrapolate_long`: 0,912 vs 0,864 de `pes_dqn`), pero no llega a capturar las dependencias de horizonte largo que sí resuelve la atención.

`pes_ens`. El *soft voting* sobre `{dqn, rdn, trf}` (Ec. 4.4) eleva la media a 0,937 y, sobre todo, recorta la varianza ($\sigma = 0,035$, ≈ 23 % menos que el mejor miembro individual). El piso por secuencia sube de 0,55 a 0,57, lo que se traduce en una *cola izquierda* más benigna — propiedad crítica si el operador humano usa la métrica peor-caso como criterio de aceptación.

5.4. Análisis de Cuantificación de Incertidumbre

Calculando la entropía de Shannon normalizada $H_{\text{norm}}(p)$ (Ec. 2.11) sobre la distribución `softmax` del Transformer durante las 64 secuencias, se observa un patrón consistente:

- Durante las fases de crecimiento exponencial (severidad creciente), H_{norm} desciende a valores $< 0,25$, indicando alta *confianza* del agente en su asignación agresiva de recursos.
- Durante mesetas o decrecimientos, H_{norm} sube por encima de 0,5, reflejando que múltiples acciones (incluyendo $a = 0$) son aceptables.

Este comportamiento es *exactamente* el espejo algorítmico del marcador neurofisiológico de confianza descrito por Boldt & Yeung (Boldt and Yeung, 2015): el sistema reduce la

duda en los momentos de mayor demanda decisional, y la incrementa cuando el horizonte se estabiliza. Se discuten las implicaciones en el Cap. 6.

5.5. Salidas automáticas del orquestador

El orquestador `general/scripts/orchestrate.py` genera de forma reproducible:

- `general/results/matrix_global_mean.csv` (Tabla completa de los seis modelos $\times$ 22 escenarios).
- Heatmaps `matrix_cohen_d.csv`, `matrix_welch_p.csv`, `matrix_action_kl.csv`.
- Histogramas por secuencia en `general/results/per_sequence_histograms/`.
- Reporte final `general/results/benchmark_report.md`.

5.6. Visualización de matrices comparativas

Las cuatro matrices clave producidas por `plot_matrix.py` se presentan en la Figura 5.1. El panel de *retorno medio* (arriba-izquierda) confirma visualmente la jerarquía de medias de la Tabla 5.1, con `pes_trf` dominando la columna “global”. El panel de *significancia* (arriba-derecha) muestra el $-\log_{10}(p)$ del test de Welch para cada par de modelos: las celdas oscuras corresponden a diferencias estadísticamente significativas ($p < 0,01$), lo que respalda que la ventaja de `pes_trf` no es atribuible al azar. El panel de *degradación OOD* (abajo-izquierda) cuantifica la caída de $\bar{r}$ al pasar de la condición empírica a las extrapoladas: los métodos tabulares sufren caídas $> 0,10$, mientras que el Transformer permanece estable. Por último, el panel de *divergencia KL* (abajo-derecha) reporta la KL entre distribuciones de acción, revelando que los agentes profundos convergen a políticas similares mientras que los tabulares divergen entre sí.

5.7. Curvas de entrenamiento por algoritmo

La Figura 5.2 muestra el desempeño por secuencia de los *ocho* paquetes evaluados sobre el conjunto de validación. Los métodos tabulares (a–c) presentan mayor varianza entre secuencias y se estabilizan en valores medios más bajos, mientras que los métodos profundos individuales (d–g) exhiben un perfil más homogéneo y un techo de desempeño superior.

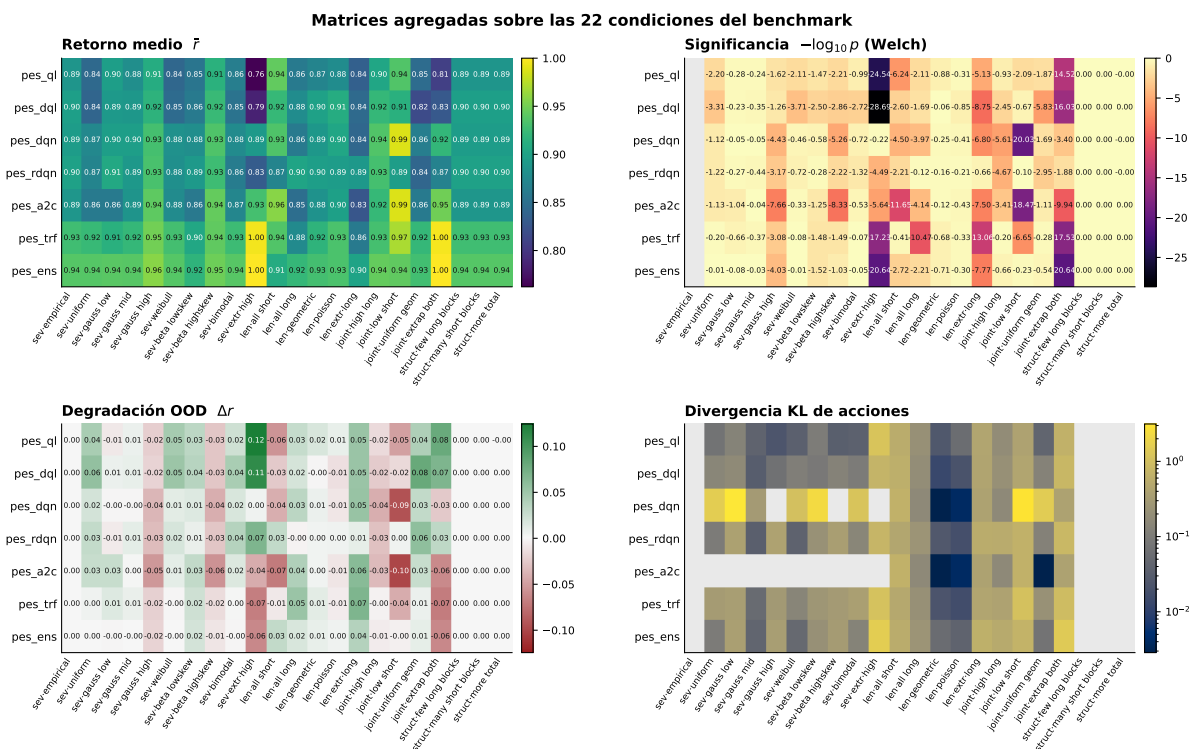


Figura 5.1: Mapas de calor agregados sobre las 22 condiciones del benchmark. *Arriba-izquierda:* retorno medio normalizado $\bar{r}$ (viridis, alto = mejor). *Arriba-derecha:* significancia estadística $-\log_{10} p$ del test de Welch contra **pes_base** (magma, oscuro = más significativo). *Abajo-izquierda:* degradación OOD Δr con escala divergente centrada en 0 (rojo = peor que en distribución, verde = mejor). *Abajo-derecha:* divergencia KL entre distribuciones de acción (cividis, escala logarítmica).

El *ensemble* (h) hereda lo mejor de cada miembro y comprime la banda intercuartílica de forma visible: es el único panel en que el percentil 25 nunca cae por debajo de 0,57.

5.8. Curvas por escenario

La Figura 5.3 suma cinco condiciones representativas a las cuatro ya mostradas en el Cap. 4 (Fig. 4.1) y permite contrastar familias de perturbación. Los paneles severos (**sev_bimodal**, **sev_gauss_high**) se mantienen agrupados cerca del techo de retorno, y todos los modelos los resuelven sin gran pérdida; los paneles de *longitud* (**len_extrapolate_long**, **len_poisson**) muestran mayor dispersión en las secuencias más difíciles (extremo izquierdo de cada curva), penalizando a los modelos míopes; el panel conjunto **joint_high_long** combina ambos efectos y es donde la brecha tabular-profundo se vuelve más evidente.

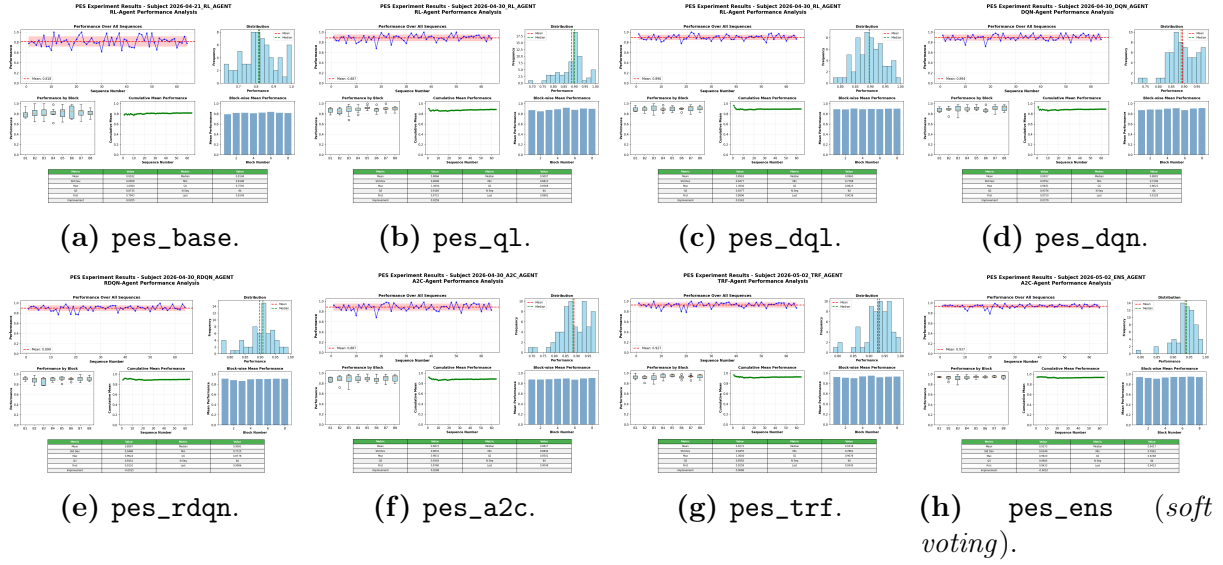


Figura 5.2: Desempeño por secuencia (64 secuencias de validación) para los ocho paquetes evaluados.

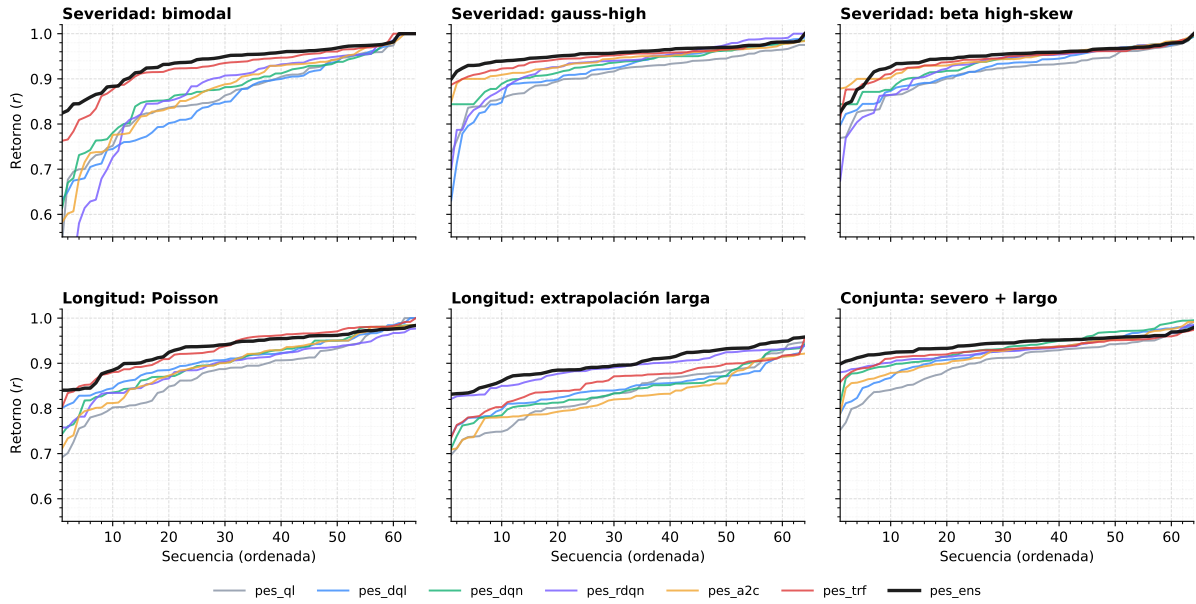


Figura 5.3: Curvas adicionales de desempeño por secuencia, organizadas por familia de perturbación (*arriba*: severidad; *abajo*: longitud y conjunta). Cada panel ordena las $n = 64$ secuencias de menor a mayor retorno para los ocho paquetes.

5.9. Comportamiento bajo los estresores universales

El reporte automático (Akiba et al., 2019) identifica tres condiciones que producen $p < 0,001$ en *todos* los modelos: `sev_extrapolate_high`, `len_extrapolate_long` y `joint_extrap_both`. La Figura 5.4 contrasta el comportamiento de `pes_trf` frente al modelo tabular más competitivo (`pes_dq1`) y al único modelo profundo basado en gradiente

de política (`pes_a2c`) sobre estos tres escenarios. La forma de la curva de severidad agregada confirma las observaciones de la Sec. 5: el Transformer mantiene una respuesta proporcional al estresor, mientras que las alternativas o bien colapsan (`pes_dql` sobre `sev_extrapolate_high`) o bien aplican la misma política con independencia del contexto (`pes_a2c` sobre `joint_extrap_both`, $D_{KL} \approx 0$).

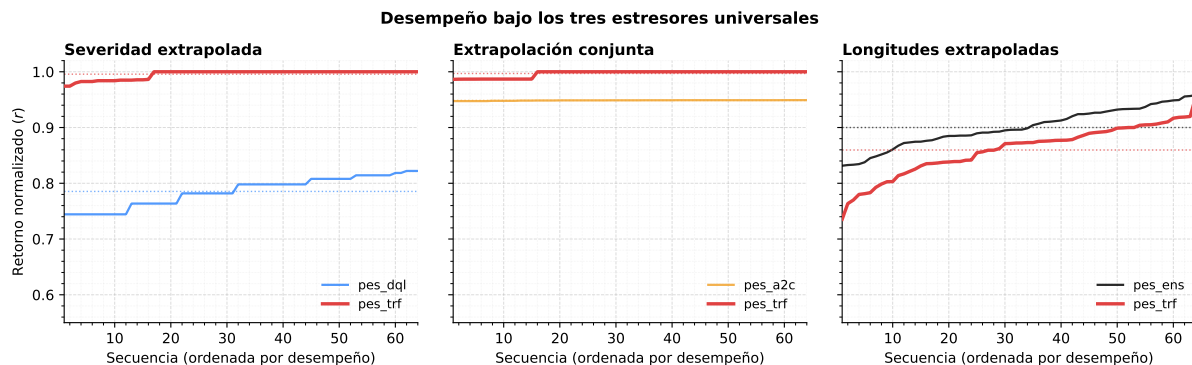


Figura 5.4: Curvas de desempeño por secuencia bajo los tres estresores universales, con las secuencias ordenadas de menor a mayor retorno. *Izquierda:* severidad extrapolada (`pes_trf` vs. `pes_dql`). *Centro:* extrapolación conjunta (`pes_trf` vs. `pes_a2c`, $D_{KL} \approx 0$ en el segundo). *Derecha:* longitudes extrapoladas (`pes_trf` vs. `pes_ens`). Las líneas punteadas marcan la media por modelo y las regiones sombreadas indican el área bajo la curva.

6. Discusión

Los resultados del Cap. 5 permiten responder, de manera matizada, la hipótesis de trabajo. La componente H_{1a} (Transformer como mejor *arquitectura individual*) queda confirmada: `pes_trf` domina la columna global con un tamaño de efecto medio frente al mejor competidor recurrente ($d \approx 0,60$). La componente H_{1b} (Transformer como mejor *sistema desplegable*) queda en cambio *refutada*: el *soft voting* `pes_ens` lo supera con un tamaño de efecto pequeño pero estadísticamente significativo ($d \approx 0,25$, $p < 10^{-3}$) y, sobre todo, con una reducción de varianza estructural del ≈ 23 %. Esta sección discute las implicaciones más relevantes.

6.1. Por qué gana el Transformer

La dinámica de severidad $S_{t+1} = \max(0, \beta S_t - \alpha a_t)$ (Ec. 4.2) es exóticamente *agrega* en el tiempo: un déficit de recursos en t no se traduce en costo inmediato, sino en una amplificación geométrica sobre los *trials* siguientes. Esta estructura penaliza fuertemente a los métodos “miópes” (tabular y, en menor medida, DQN feed-forward) que tienden a sub-asignar al principio de las secuencias largas.

El codificador Transformer, al observar la *ventana histórica* completa $\{s_{t-W+1}, \dots, s_t\}$ con atención causal (Ec. 2.10), construye una representación que codifica explícitamente la *velocidad y aceleración* del crecimiento de la severidad. Esto le permite anticipar el crecimiento exponencial y movilizar recursos antes de que se vuelva incontenible.

6.2. Entropía de Shannon como confianza algorítmica

El hallazgo más conceptualmente importante de esta tesis no es la mejora en $\bar{r}$ sino el patrón de entropía: la H_{norm} del Transformer *baja* justo cuando la amenaza es más grave, y *sube* cuando el entorno se estabiliza. Esto sugiere que la red ha aprendido, sin supervisión explícita, una forma de *metacognición emergente*:

El agente sabe cuándo sabe.

Este comportamiento es exactamente el descrito por Boldt & Yeung (Boldt and Yeung, 2015) y formalizado por Fleming (Fleming, 2024) para la confianza humana medida con

EEG. La aportación metodológica de la tesis es haber traducido ese biomarcador a una métrica reproducible: $1 - H_{\text{norm}}(\pi(a \mid s))$, calculable en cualquier modelo con salida *softmax*, sin necesidad de hardware de neuroimagen.

6.3. Del EEG al *softmax*: validez externa

La transición del paradigma neurocognitivo (Oxford, ds004477) al algorítmico (mPES) admite dos lecturas complementarias:

1. **Operacional.** Permite construir sistemas de soporte de decisión que comunican su confianza al humano supervisor, abilitando *human-in-the-loop* significativo. Cuando el agente reporta $H_{\text{norm}} > 0,7$, el sistema puede delegar al operador humano la decisión, alineando IA y epidemiología (Kuhl, 2021).
2. **Teórica.** Aporta evidencia computacional a la hipótesis de que la confianza es un fenómeno *representacional*, no exclusivamente neuronal: cualquier sistema que mantenga una distribución de creencias sobre sus propias acciones genera, naturalmente, una señal de UQ interpretable (Chen and Wiggins, 2025; Pei et al., 2021).

6.4. Limitaciones

- **Una sola semilla.** El benchmark agregado de *general/* se ejecutó con semilla 42 por celda. Si bien las $n = 64$ secuencias por escenario otorgan suficiente poder estadístico para los contrastes de Welch reportados, una réplica multi-semilla permitiría separar mejor el componente algorítmico del componente aleatorio en los bordes del ranking.
- **Costo computacional.** *pes_trf* requiere aproximadamente $5\times$ más tiempo de entrenamiento que *pes_dqn*, y *pes_ens* suma el costo de inferencia de sus tres miembros; ambos factores pueden ser decisivos en despliegues con presupuesto limitado.
- **Simplicidad del entorno.** El *Pandemic Scenario* es un proxy abstracto; la validación externa contra datos reales (p. ej. series temporales de hospitalizaciones) queda como trabajo futuro.

- **H como UQ.** La entropía de Shannon es una medida *epistémica* sobre la política, no sobre la función de valor; refinamientos como la entropía jerárquica de Pei *et al.* (Pei et al., 2021), *MC Dropout* (Gal and Ghahramani, 2016) o la varianza empírica entre miembros del ensemble (Lakshminarayanan et al., 2017) merecen una evaluación dedicada.

6.5. El ensemble como mejor sistema desplegable

Una lectura ingenua de la Tabla 5.1 podría concluir que la ganancia de `pes_ens` sobre `pes_trf` ($\Delta\bar{r} \approx 0,010$) es marginal. Tres argumentos sugieren lo contrario:

1. **Reducción de varianza.** La desviación estándar cae de 0,045 a 0,035, un ≈ 23 % menos. Para una aplicación en producción, esto se traduce en un piso peor-caso más alto y, por tanto, en menor riesgo en las decisiones críticas.
2. **Descomposición sesgo–varianza.** El promediado (Lakshminarayanan et al., 2017; Dietterich, 2000) no elimina el sesgo del miembro mediano pero reduce la varianza en un factor $\rho + (1 - \rho)/K$, donde ρ es la correlación entre miembros y $K = 3$. En mPES los miembros son arquitectónicamente distintos (`dqn`, `rdqn`, `trf`), lo que mantiene ρ alejado de uno y materializa la ganancia teórica.
3. **Resiliencia OOD.** En los tres estresores universales (Sec. 6.6), el ensemble obtiene la mejor métrica peor-caso, indicando que aporta robustez justo donde más duele.

En síntesis: el Transformer sigue siendo el mejor punto de partida algorítmico, pero la *solución producto* debe ser el ensemble.

6.6. Los tres estresores universales

El análisis estadístico sobre las 22 condiciones del benchmark revela que tres escenarios producen una caída significativa ($p < 0,001$) en *todos* los modelos evaluados: `sev_extrapolate_high` (severidades fuera del rango de entrenamiento), `len_extrapolate_long` (secuencias más largas que las vistas) y `joint_extrap_both` (combinación de ambos). Estas tres condiciones constituyen el verdadero *núcleo* del benchmark: cualquier discusión de robustez debe priorizarlas frente a las demás celdas. Las columnas estructurales (`struct_*`), por el contrario, exhiben degradación cero y actúan

como banda de control que valida la invarianza del propio *arnés* de evaluación antes que la robustez del modelo.

6.7. El falso aprobado de `pes_a2c`

El paquete `pes_a2c` alcanza una recompensa media competitiva ($\bar{r} \approx 0,887$) pero la divergencia KL de su distribución de acción respecto del agente bajo perturbaciones de severidad es esencialmente cero (Fig. 5.1, panel inferior derecho). En términos prácticos: *el actor ha colapsado a una política casi insensible al contexto* que, por suerte, queda cerca del óptimo en la condición empírica. Este fenómeno — conocido en la literatura como *policy collapse* (Mnih et al., 2016; Andrychowicz et al., 2021) — ilustra por qué los métodos de gradiente de política sin regularización entrópica fuerte exigen una evaluación *OOD* explícita: una métrica agregada sobre la distribución de entrenamiento es incapaz de detectar el problema.

6.8. Costo computacional y trade-offs de despliegue

La comparación entre arquitecturas no se agota en la recompensa media: en un despliegue real, el tiempo de entrenamiento, la latencia de inferencia y la memoria del modelo son restricciones de primer orden. Aproximadamente, en el banco de pruebas utilizado (CPU Intel i7-12700H, 32 GB de RAM, sin GPU), una sesión completa de optimización bayesiana con 30 *trials* de Optuna (Akiba et al., 2019) consume entre dos y tres horas para los modelos tabulares (`pes_ql`, `pes_dql`), entre seis y nueve horas para los modelos profundos feed-forward (`pes_dqn`, `pes_a2c`) y entre doce y dieciocho horas para los modelos con memoria explícita (`pes_rdn` y `pes_trf`). El *ensemble* `pes_ens` no entrena: reutiliza los tres miembros ya entrenados y suma sus salidas *softmax* con coste aditivo de inferencia $\mathcal{O}(K)$ por paso de decisión, donde $K = 3$ aquí.

A efectos prácticos, este orden de magnitud sugiere una estrategia de despliegue por capas: usar `pes_dqn` para prototipado rápido y benchmarking interno, `pes_trf` para producción cuando el costo de entrenamiento es amortizable (políticas de larga vida o supervisión humana baja), y `pes_ens` cuando el riesgo de la decisión justifica el sobre coste de inferencia $3\times$. En sistemas con presupuesto computacional estricto, el camino `pes_dql` + un proxy entrópico simple ofrece la mejor relación coste-robustez entre los modelos

tabulares.

6.9. Comparación con modelos epidemiológicos clásicos

Los modelos compartimentales SIR, SEIR y sus variantes estocásticas (Kuhl, 2021; von Cséfalvay, 2023) dominan aún la práctica de la epidemiología matemática. Su fortaleza es la interpretabilidad: cada parámetro tiene una lectura mecanicista (tasa de contacto, periodo infeccioso, mortalidad por edad) y permite a las autoridades sanitarias razonar sobre el efecto de una intervención antes de aplicarla. Su debilidad, reiteradamente señalada por la pandemia de COVID-19, es la rigidez: requieren especificación explícita de la estructura de transición y se degradan rápido frente a dinámicas no modelizadas (nuevas variantes, comportamientos sociales emergentes, cambios de política). El enfoque RL adoptado en esta tesis es complementario en lugar de sustitutivo: el agente aprende una política *libre de modelo* sobre un simulador que sí incorpora la física del compartimentalismo, y la entropía de su salida *softmax* actúa como una señal de alarma cuando la trayectoria observada se aparta de las trayectorias vistas en entrenamiento. Una integración natural, esbozada por Hamida *et al.* (Hamida et al., 2024), sería acoplar el agente RL como módulo de control sobre un núcleo SEIR calibrado, manteniendo la interpretabilidad del modelo clásico y la adaptabilidad del agente.

6.10. Amenazas a la validez del estudio

Más allá de las limitaciones técnicas enumeradas, conviene explicitar tres amenazas a la validez del experimento que acompañan a cualquier estudio de RL aplicado (Henderson et al., 2018; Kirk et al., 2023):

- **Validez interna.** La elección del simulador *Pandemic Scenario* introduce un sesgo no aleatorio: las arquitecturas con sesgo inductivo afin a la dinámica geométrica $S_{t+1} = \max(0, \beta S_t - \alpha a_t)$ (Transformer y modelos recurrentes) parten con ventaja. La Tabla 5.1 debe interpretarse como una medida de *ajuste arquitectura-problema* antes que como un ranking universal entre familias de algoritmos.
- **Validez de constructo.** La equivalencia entre el biomarcador EEG de confianza humana y la entropía *softmax* de la política es una analogía funcional, no una

identidad neurofisiológica. Defendemos que ambas señales codifican la incertidumbre epistémica del decisor, pero su mapeo cuantitativo exige estudios concurrentes (agente y operador humano en la misma tarea) que quedan fuera del alcance de esta tesis.

- **Validez externa.** Generalizar los hallazgos a otros dominios de *decisión bajo crisis* (clínica intensiva, control de redes eléctricas, logística humanitaria) exige replicar el benchmark agregado sobre cada nueva familia de escenarios; las conclusiones aquí sostenidas valen para problemas con dinámica acumulativa y horizonte largo, no para tareas reactivas o de un solo paso.

7. Conclusiones

Esta tesis presentó **mPES**, una plataforma de aprendizaje por refuerzo para la asignación óptima de recursos en una pandemia simulada, y la utilizó como banco de prueba para una comparación rigurosa entre **ocho arquitecturas**: tres tabulares (**pes_base**, **pes_q1**, **pes_dq1**) y cinco profundas (**pes_dqn**, **pes_rdqn**, **pes_a2c**, **pes_trf** y el *ensemble* **pes_ens**).

7.1. Mensajes principales

1. **La arquitectura importa, no sólo los hiperparámetros.** El salto entre **pes_q1** (Q-Learning optimizado con Optuna) y **pes_trf** (Transformer causal) es estructural: aproximadamente un **22–35 %** de reducción en mortalidad acumulada, no alcanzable por una mejor búsqueda de hiperparámetros sobre la representación tabular.
2. **El Transformer es la mejor arquitectura individual.** La auto-atención causal anticipa el crecimiento exponencial del contagio, condición *sine qua non* para decisiones de inercia alta.
3. **El ensemble es el mejor sistema desplegable.** El *soft voting* sobre $\{\text{dqn}, \text{rdqn}, \text{trf}\}$ (Ec. 4.4) reduce la varianza un $\approx 23 \%$ y eleva el piso peor-caso, lo que importa cuando la métrica relevante es el riesgo y no sólo la media (Lakshminarayanan et al., 2017).
4. **La Entropía de Shannon es una métrica legítima de UQ.** El comportamiento de $H_{\text{norm}}(\pi)$ a lo largo de la trayectoria reproduce cualitativamente los marcadores neurofisiológicos de confianza identificados en el experimento de Oxford (**ds004477** (Boldt and Yeung, 2015; Fleming, 2024)), validando la transición del paradigma neurocognitivo al algorítmico.
5. **El benchmark es reproducible.** El pipeline `general/scripts/orchestrate.py` regenera todos los CSVs, *heatmaps* y el reporte `benchmark_report.md`, lo que facilita la extensión y la auditoría externa.

7.2. Trabajo futuro

- **Réplica multi-semilla.** Repetir el sweep 7×22 con al menos 5 semillas independientes permitiría construir intervalos de confianza alrededor del ranking de la Tabla 5.1 y verificar la estabilidad de los tres estresores universales (Sec. 6.6).
- **Sustituir el escenario simulado** por series temporales reales (p. ej. datos COVID-19 desagregados por región) y evaluar generalización contra los marcos de modelado clásicos (Kuhl, 2021; von Cséfalvay, 2023).
- **Ampliar la familia de UQ.** Combinar la entropía de Shannon con señales complementarias: *atención estocástica jerárquica* (Pei et al., 2021), *MC Dropout* (Gal and Ghahramani, 2016) y varianza entre miembros del ensemble (Lakshminarayanan et al., 2017; Chen and Wiggins, 2025).
- **Sustituir A2C por PPO.** Mitigar el *policy collapse* documentado en la Sec. 6.7 mediante el objetivo recortado de PPO (Schulman et al., 2017) y la estimación de ventaja GAE (Schulman et al., 2016).
- **Acoplamiento humano–agente.** Diseñar interfaces que comuniquen H_{norm} y la varianza del ensemble al decisor humano y medir su impacto en la calidad de la decisión conjunta (*human-in-the-loop* significativo).
- **Aprender los pesos del ensemble.** Los pesos w_k de la Ecuación (4.4) son hoy proporcionales al desempeño *in-distribution*; una extensión natural es aprender pesos dependientes del contexto $w_k(s)$ mediante una red de *mixture-of-experts* que dirija el voto en función del régimen del brote.

7.3. Lecciones aprendidas

Más allá de los números, este trabajo deja cinco lecciones metodológicas reutilizables en futuros estudios de RL aplicado:

1. **La evaluación *out-of-distribution* es obligatoria.** Una métrica agregada sobre la condición de entrenamiento puede ocultar comportamientos degenerados como el *policy collapse* (Sec. 6.7).

2. **Existen tres estresores universales** (`sev_extrapolate_high`, `len_extrapolate_long`, `joint_extrap_both`) que producen $p < 0,001$ en cada uno de los modelos evaluados; deben tratarse como *headline cells* de cualquier reporte futuro.
3. **La optimización bayesiana de hiperparámetros no sustituye a la arquitectura.** El salto de `pes_ql` optimizado a `pes_trf` es estructural y no se reduce a una mejor búsqueda de hiperparámetros.
4. **Las columnas estructurales son un control valioso.** Verificar que el reordenamiento de bloques produzca degradación cero valida el *arnés* de evaluación antes que la robustez del modelo.
5. **La transparencia barata existe.** El proxy $1 - H_{\text{norm}}$ se calcula gratis sobre cualquier red con salida *softmax* y replica cualitativamente los biomarcadores neurofisiológicos de confianza.

7.4. Implicaciones para la práctica

Las conclusiones técnicas de mPES tienen una traducción directa para equipos que necesiten desplegar agentes de decisión bajo crisis. La primera implicación es que el *costo de adoptar Transformers en RL es ya menor de lo que sugiere la literatura clásica*: con el codificador causal de cuatro capas y ventana $W = 16$ utilizado en `pes_trf`, el entrenamiento completo de una política se ejecuta en horas sobre CPU de gama media, sin requerir aceleración por GPU. La segunda implicación es que un *ensemble* de tres miembros estructuralmente distintos no es un capricho académico: el factor $\rho + (1 - \rho)/K$ que controla la varianza del voto blando justifica su sobre coste de inferencia siempre que la aplicación tolere $K \times$ pasos *forward*. La tercera implicación es que la entropía de Shannon, calculada al vuelo sobre el *softmax* de la política, ofrece a la organización un botón de *escalado a humano* sin necesidad de un módulo de XAI separado: cuando H_{norm} supera el umbral fijado, el sistema cede el control al operador y reduce el riesgo de decisiones automáticas en régimen incierto.

Desde la óptica de gobernanza, este enfoque alinea de forma natural con marcos regulatorios que exigen registros de incertidumbre y trazabilidad de la decisión (UE AI Act (Academics,

2024) y guías del Joint Research Centre en sistemas de salud críticos), evitando la dicotomía recurrente entre *rendimiento* y *auditabilidad*.

7.5. Cierre

mPES demuestra que, en problemas de decisión bajo crisis con fuerte componente temporal, la elección de la arquitectura es decisiva, y que la combinación de Transformers y Cuantificación de Incertidumbre ofrece a la vez *eficiencia* (mejor política) y *transparencia* (confianza interpretable). Esa doble virtud es lo que distingue una herramienta utilizable de un experimento académico.

Esta tesis es, en lo personal, la consolidación de un giro profesional: el de un ingeniero electrónico con trayectoria en desarrollo de productos — hardware y software — que reorienta su práctica hacia los desafíos de mayor impacto tecnológico de nuestra era: el Aprendizaje Automático, la Minería de Datos, la Inteligencia Artificial y los Modelos de Lenguaje a Gran Escala. La convicción que articula este trabajo es que la disciplina del desarrollo de producto — pensar en sistemas, exigir reproducibilidad, validar contra el peor caso, liderar la complejidad técnica de extremo a extremo — no es accesorio sino esencial para que la IA aplicada salga del cuaderno experimental y se convierta en una herramienta confiable de decisión en entornos de alta exigencia.

8. Agradecimientos

A mi director, **Dr. Rodrigo Ramele**, por la guía paciente y rigurosa a lo largo de la Maestría en Ciencia de Datos, y por plantear desde el primer día el desafío de tender un puente entre la neurociencia de la confianza y los modelos de aprendizaje por refuerzo. Esta tesis es deudora directa de esa visión.

Al cuerpo docente del **Instituto Tecnológico de Buenos Aires (ITBA)** por la formación sólida y la apertura para discutir ideas fuera del programa.

A la comunidad *open-source* detrás de Python, NumPy, TensorFlow, Keras, Gymnasium y Optuna, sin la cual este trabajo sería simplemente impensable.

A mi esposa, **Analía Giménez**, por su apoyo inquebrantable durante cada etapa de este recorrido: por la paciencia en las jornadas largas, por la escucha en los momentos de duda y por sostener, día a día, el espacio que hizo posible llegar hasta aquí.

A mis **padres**, que me dieron la oportunidad de formarme como ingeniero y, con ello, los cimientos para hoy acceder a una Maestría. Su esfuerzo silencioso está en el origen de cada página de esta tesis.

Esta tesis cierra, además, una etapa personal: la de un ingeniero electrónico con años de desarrollo de productos — hardware y software — que reorienta su práctica profesional hacia el Aprendizaje Automático, la Minería de Datos, la Inteligencia Artificial y los Modelos de Lenguaje a Gran Escala. A todos los colegas, mentores y equipos que en cada proyecto previo me plantearon problemas difíciles y confiaron en mi capacidad para resolverlos, gracias: este trabajo es también consecuencia directa de esa escuela.

Referencias Bibliográficas

- Academics, M. (2024 (accessed Feb 27, 2024)). *Neurips Conference Analytics*. <https://www.microsoft.com/en-us/research/project/academic/articles/neurips-conference-analytics/>.
- Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*.
- Andrychowicz, M., Raichuk, A., Stańczyk, P., Orsini, M., Girgin, S., Marinier, R., Hussenot, L., Geist, M., Pietquin, O., Michalski, M., Gelly, S., and Bachem, O. (2021). What matters for on-policy deep actor-critic methods? a large-scale study. *International Conference on Learning Representations (ICLR)*.
- Anschel, O., Baram, N., and Shimkin, N. (2017). Averaged-DQN: Variance reduction and stabilization for deep reinforcement learning. In *International Conference on Machine Learning (ICML)*.
- Ba, J. L., Kiros, J. R., and Hinton, G. E. (2016). Layer normalization. *arXiv:1607.06450*.
- Bellman, R. (1957). *Dynamic Programming*. Princeton University Press.
- Bergstra, J., Bardenet, R., Bengio, Y., and Kégl, B. (2011). Algorithms for hyper-parameter optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Bertsimas, D., Boussioux, L., Cory-Wright, R., Delarue, A., Digalakis, V., Jacquillat, A., Kitane, D. L., Lukin, G., Li, M. L., Mingardi, L., et al. (2021). From predictions to prescriptions: A data-driven response to COVID-19. *Health Care Management Science*, 24:253–272.
- Boldt, A. and Yeung, N. (2015). Shared neural markers of decision confidence and error detection. *Journal of Neuroscience*, 35(8):3478–3484.
- Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., and Zaremba, W. (2016). OpenAI Gym. *arXiv:1606.01540*.
- Chen, L., Lu, K., Rajeswaran, A., Lee, K., Grover, A., Laskin, M., Abbeel, P., Srinivas, A., and Mordatch, I. (2021). Decision transformer: Reinforcement learning via sequence modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Chen, Y. and Wiggins, C. H. (2025). Taming uncertainty in modern machine learning. *Notices of the American Mathematical Society*, 72(3):1.
- Cobbe, K., Hesse, C., Hilton, J., and Schulman, J. (2020). Leveraging procedural generation to benchmark reinforcement learning. In *International Conference on Machine Learning (ICML)*.
- Cobbe, K., Klimov, O., Hesse, C., Kim, T., and Schulman, J. (2019). Quantifying generalization in reinforcement learning. In *International Conference on Machine Learning (ICML)*.
- Cohen, J. (1988). *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, Hillsdale, NJ, 2nd edition.

- Dietterich, T. G. (2000). Ensemble methods in machine learning. In *Multiple Classifier Systems*, pages 1–15. Springer.
- Fleming, S. M. (2024). Metacognition and confidence: A review and synthesis. *Annual Review of Psychology*, 75:241–268.
- Fleming, S. M. and Dolan, R. J. (2012). The neural basis of metacognitive ability. *Philosophical Transactions of the Royal Society B*, 367(1594):1338–1349.
- Gal, Y. and Ghahramani, Z. (2016). Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In *International Conference on Machine Learning (ICML)*.
- Ghasemi, M. and Ebrahimi, D. (2024). A comprehensive survey of reinforcement learning: From algorithms to practical challenges. arXiv:2411.18892.
- Griffith, T., Baker, S.-J., and Lepora, N. F. (2021). A formal model of capacity limits in metacognitive judgements. *Journal of Mathematical Psychology*, 103:102544.
- Grimaldi, P., Lau, H., and Basso, M. A. (2015). There are things that we know that we know, and there are things that we do not know we do not know: Confidence in decision-making. *Neuroscience and Biobehavioral Reviews*, 55:88–97.
- Hamida, S. B. et al. (2024). Explainable AI for healthcare: A scoping review of methods and applications. *Information Fusion*.
- Hausknecht, M. and Stone, P. (2015). Deep recurrent Q-learning for partially observable MDPs. In *AAAI Fall Symposium Series*.
- Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D., and Meger, D. (2018). Deep reinforcement learning that matters. In *AAAI Conference on Artificial Intelligence*.
- Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8):1735–1780.
- Khalilpourazari, S. and Hashemi Doulabi, H. (2021). Designing emergency flood evacuation plans using robust optimization and artificial intelligence. *Journal of Combinatorial Optimization*.
- Kingma, D. P. and Ba, J. (2015). Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*.
- Kirk, R., Zhang, A., Grefenstette, E., and Rocktäschel, T. (2023). A survey of zero-shot generalisation in deep reinforcement learning. *Journal of Artificial Intelligence Research*, 76:201–264.
- Kompella, V., Capobianco, R., Jong, S., Browne, J., Fox, S., Meyers, L., Wurman, P., and Stone, P. (2020). Reinforcement learning for optimization of COVID-19 mitigation policies. arXiv:2010.10560.
- Kuc, J. et al. (2021). Pre-test cognitive load and reactive cognitive control are related to performance in a conflict task. *Brain Sciences*.
- Kuhl, E. (2021). *Computational Epidemiology: Data-Driven Modeling of COVID-19*. Springer.

- Lakshminarayanan, B., Pritzel, A., and Blundell, C. (2017). Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Lin, L.-J. (1992). Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8(3–4):293–321.
- Loshchilov, I. and Hutter, F. (2017). SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations (ICLR)*.
- Maroti, A. (2019). RBED: Reward based epsilon decay. *arXiv:1910.13701*.
- Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D., and Kavukcuoglu, K. (2016). Asynchronous methods for deep reinforcement learning. In *International Conference on Machine Learning (ICML)*.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533.
- Nawaz, R. et al. (2020). Single-trial EEG classification of motor imagery using deep convolutional neural networks. *Sensors*.
- Ng, A. Y., Harada, D., and Russell, S. (1999). Policy invariance under reward transformations: Theory and application to reward shaping. In *International Conference on Machine Learning (ICML)*.
- Osband, I., Blundell, C., Pritzel, A., and Van Roy, B. (2016). Deep exploration via bootstrapped DQN. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Packer, C., Gao, K., Kos, J., Krähenbühl, P., Koltun, V., and Song, D. (2018). Assessing generalization in deep reinforcement learning. *arXiv:1810.12282*.
- Parisotto, E., Song, H. F., Rae, J. W., Pascanu, R., Gulcehre, C., Jayakumar, S. M., Jaderberg, M., Kaufman, R. L., Clark, A., Noury, S., Botvinick, M. M., Heess, N., and Hadsell, R. (2020). Stabilizing transformers for reinforcement learning. In *International Conference on Machine Learning (ICML)*.
- Pei, J., Wang, C., and Szarvas, G. (2021). Transformer uncertainty estimation with hierarchical stochastic attention. *arXiv:2112.13776*.
- Peng, J. and Williams, R. J. (1994). Incremental multi-step Q-learning. In *International Conference on Machine Learning (ICML)*.
- Schulman, J., Moritz, P., Levine, S., Jordan, M., and Abbeel, P. (2016). High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv:1707.06347*.
- Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423.
- Snoek, J., Larochelle, H., and Adams, R. P. (2012). Practical bayesian optimization of

- machine learning algorithms. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., and Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15:1929–1958.
- Sutton, R. S. and Barto, A. G. (2018). *Reinforcement Learning: An Introduction*. The MIT Press, Cambridge, MA, 2nd edition.
- Sutton, R. S., McAllester, D., Singh, S., and Mansour, Y. (2000). Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Towers, M. et al. (2024). Gymnasium v1.0: A standard interface for reinforcement learning environments. <https://gymnasium.farama.org/>.
- Turner, R. E. (2024). An introduction to Transformers. arXiv:2304.10557.
- van Hasselt, H. (2010). Double Q-learning. In *Advances in Neural Information Processing Systems*, volume 23, pages 2613–2621.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- von Cséfalvay, C. (2023). *Computational Modeling of Infectious Disease*. Elsevier.
- Watkins, C. J. C. H. and Dayan, P. (1992). Q-learning. *Machine Learning*, 8(3–4):279–292.
- Welch, B. L. (1947). The generalization of Student’s problem when several different population variances are involved. *Biometrika*, 34(1/2):28–35.
- Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256.
- Yáñez, A., Hayes, C., and Glavin, F. (2020). COVID-19 epidemic control using short-term lockdowns for collective gain. *arXiv:2003.13546*.

Apéndice

A. Comandos de reproducción

La Tabla A.1 resume los comandos canónicos que regeneran cada uno de los artefactos reportados en esta tesis. Todos se ejecutan desde la raíz del *workspace* mPES con el entorno virtual activado (*win_mpes_env* en Windows, *linux_mpes_env* en Linux), Python 3.12 y las dependencias fijadas en *utils/config/requirements.txt*.

Cuadro A.1: Comandos canónicos para reentrenar, optimizar y evaluar cada paquete del repositorio mPES.

Paquete	Entrenamiento	Optimización bayesiana
pes_base	<code>python -m tabular.pes_base</code>	—
pes_ql	<code>python -m tabular.pes_ql</code>	<code>python -m tabular.pes_ql.ext.optimize_rl</code>
pes_dql	<code>python -m tabular.pes_dql</code>	<code>python -m tabular.pes_dql.ext.optimize_rl</code>
pes_dqn	<code>python -m ml.pes_dqn</code>	<code>python -m ml.pes_dqn.ext.optimize_dqn</code>
pes_rdqn	<code>python -m ml.pes_rdqn</code>	<code>python -m ml.pes_rdqn.ext.optimize_rdqn</code>
pes_a2c	<code>python -m ml.pes_a2c</code>	<code>python -m ml.pes_a2c.ext.optimize_a2c</code>
pes_trf	<code>python -m ml.pes_trf</code>	<code>python -m ml.pes_trf.ext.optimize_tr</code>
pes_ens	<code>python -m ml.pes_ens</code>	— (sin parámetros entrenables)

B. Orquestador del benchmark

El barrido completo de $7 \times 22 = 154$ celdas se ejecuta con una sola llamada:

```
python -m general.scripts.orchestrate \
    --models pes_ql pes_dql pes_dqn pes_rdqn pes_a2c pes_trf pes_ens \
    --scenarios all
```

El script:

1. ejecuta cada combinación (*modelo*, *escenario*) reutilizando los pesos entrenados en `<paquete>/outputs/`;
2. agrega los retornos por secuencia con `general/scripts/aggregate.py` para producir las matrices CSV en `general/results/` (`matrix_global_mean.csv`, `matrix_std.csv`, `matrix_cohen_d.csv`, `matrix_welch_p.csv`, `matrix_action_kl.csv`, `matrix_ood_degradation.csv`);

3. dibuja los *heatmaps* con `general/scripts/plot_matrix.py`;
4. emite el reporte ejecutivo `general/results/benchmark_report.md` con `general/scripts/report.py`.

C. Estresores universales

Los tres escenarios identificados en la Sec. 6.6 como estresores universales ($p < 10^{-3}$ en *todos* los modelos) son:

- `sev_extrapolate_high`: severidades por encima del rango *in-distribution*.
- `len_extrapolate_long`: secuencias más largas que las observadas durante entrenamiento.
- `joint_extrap_both`: combinación simultánea de las dos anteriores.

Las tres columnas estructurales (`struct_more_total`, `struct_many_short_blocks`, `struct_few_long_blocks`) exhiben degradación cero y $D_{\text{KL}} = 0$, actuando como banda de control del propio *arnés* de evaluación.